UNIVERSIDADE LaSalle
Viver é evoluir.

NOVA GRADUAÇÃO UNIVERSIDADE LA SALLE

UNIDADE 1

Requisitos de SOFTWARE



INOVAÇÃO E TECNOLOGIA



INOVAÇÃO
E TECNOLOGIA

unidade
1

Fundamentos da Engenharia de Requisitos

Prezado estudante,

Estamos começando uma unidade desta disciplina. Os textos que a compõem foram organizados com cuidado e atenção, para que você tenha contato com um conteúdo completo e atualizado tanto quanto possível. Leia com dedicação, realize as atividades e tire suas dúvidas com os tutores. Dessa forma, você, com certeza, alcançará os objetivos propostos para essa disciplina.

Objetivo Geral

Aprender os fundamentos da engenharia de requisitos e técnicas para elicitação de requisitos.

Editora LaSalle

Parte 1

Fundamentos da Engenharia de Requisitos

O conteúdo deste livro é disponibilizado por SAGAH.

SOLUÇÕES EDUCACIONAIS INTEGRADAS

Fundamentos da engenharia de requisitos

Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Discutir os problemas advindos dos requisitos em projetos de *software*.
- Reconhecer a importância da engenharia de requisitos no ciclo de vida de um *software*.
- Identificar as etapas da engenharia de requisitos.

Introdução

É inquestionável a presença da tecnologia em nosso dia a dia. Produtos de *software* fazem parte de praticamente todas as atividades humanas, sejam elas pessoais, sejam profissionais. Quando um *software* não funciona de forma adequada, sentimos o impacto por meio de transtornos, como a falta de sincronização de semáforos nas grandes cidades, o atraso e o cancelamento de voos, a impossibilidade de fazer uma compra *on-line*, a indisponibilidade de serviços bancários, e mais uma infinidade de contratempos. Falhas em *software* também podem tirar vidas.

Uma das principais fontes de problemas em produtos de *software* está relacionada aos requisitos. Requisitos mal compreendidos, mal especificados e mal gerenciados podem comprometer o desempenho de produtos de *software* e representam uma das maiores causas de fracasso em projetos de tecnologia da informação.

Neste capítulo, você vai ler sobre a importância do *software* na sociedade moderna e os impactos provenientes de problemas relacionados a requisitos de *software*. Verá também a importância da engenharia de requisitos no ciclo de desenvolvimento de *software* e sua influência nas atividades de desenvolvimento. Por fim, você vai saber como a engenharia de requisitos pode ajudar a eliminar, ou pelo menos minimizar, os problemas relacionados a requisitos.

1 Problemas advindos dos requisitos de *software*

Produtos de *software* estão presentes em todos os momentos de nossas vidas. Diariamente, mesmo sem nos darmos conta, entramos em contato com diferentes tipos de *software*. Acordamos despertados por uma funcionalidade de nossos *smartphones* e olhamos a previsão do tempo disponibilizada por outro aplicativo. Saímos para o exercício matinal acompanhados do *smartwatch*, que registra todo o nosso desempenho, incluindo o tempo, o percurso e até mesmo os batimentos cardíacos. Retornamos e aquecemos o café da manhã no micro-ondas, cuja programação automática é feita por um *software* embarcado no eletrodoméstico. Em seguida pedimos o transporte, utilizando mais um aplicativo no celular.

Ao chegar no trabalho, um mundo de diferentes sistemas de *software* nos apoia no dia a dia da empresa (*email*, mensagens instantâneas, programas de gestão, de controle de produção, de *marketing*, de vendas e por aí vai). Ao final do dia, cansados, voltamos do trabalho ouvindo uma música para relaxar usando o *streamer* de músicas e aproveitamos para, no caminho, pedir uma refeição no sistema de entrega de comida e acionar o ar condicionado de casa, para que esteja na temperatura ideal quando chegarmos. Ainda dá para marcar aquele encontro com os amigos para o final de semana, usando o aplicativo de troca de mensagens. Checamos nosso saldo no banco e verificamos se o débito automático da conta de luz foi realizado. Após o jantar, que pagamos com a máquina de cartão de crédito por aproximação do celular, encerramos o dia assistindo nossa série preferida no programa de *streaming* de vídeo e damos uma espiadinha nas redes sociais. O despertador já está programado para o próximo dia.

Praticamente nada disso seria possível sem o uso da tecnologia e dos sistemas de *software* que viabilizam esses serviços. Atualmente, todos os segmentos da atividade humana são permeados por *software* — e serão cada vez mais. Imagine, por um momento, se todo *software* do planeta parasse de funcionar. Quais seriam os impactos? Que atividades você realiza hoje que não poderia realizar? Que serviços essenciais deixariam de ser prestados? Algumas dessas facilidades estão tão enraizadas em nossas vidas que parece que elas sempre estiveram lá.

Com tantas novas tecnologias a cada dia, construir *software* se tornou uma atividade cada vez mais complexa. Se, por um lado, não temos mais as restrições de memória e de poder de processamento que tínhamos na década de 1960, a variedade e a complexidade dos elementos envolvidos atualmente

trazem novos desafios para o desenvolvimento de *software*. Além disso, os ambientes de negócios se tornaram mais sofisticados e complexos. O resultado é que desenvolver projetos de *software* não é uma atividade trivial, e os desafios começam na parte inicial do ciclo de vida: os **requisitos**.

Primeiramente, vamos definir requisitos. O *Glossário padrão de terminologia de engenharia de software* do *Institute of Electrical and Electronic Engineers* (IEEE, 1990) define requisito como:

1. Uma condição ou capacidade necessária para um usuário resolver um problema ou alcançar um objetivo.
2. Uma condição ou capacidade que deve ser atendida ou tida por um sistema ou componente do sistema para satisfazer a um contrato, padrão, especificação ou outro documento formalmente imposto.
3. Uma representação documentada de uma condição ou capacidade conforme estabelecido em 1 e 2.

Independentemente de a empresa ter processos mais burocráticos e formais de desenvolvimento ou utilizar métodos ágeis, requisitos mal compreendidos e mal gerenciados são causa frequente de inúmeros prejuízos e insatisfações. Diversos são os exemplos de falhas de *software* que podem ser ligadas a problemas relacionados a requisitos e levar a milhões em prejuízos. Vamos dar uma olhadinha em algumas delas?

- Na década de 1990, o Aeroporto de Denver, no Colorado, havia sido projetado para ser o maior *hub* de aviação dos Estados Unidos. Um dos pontos mais críticos estava relacionado aos requisitos que o sistema de gerenciamento de bagagens deveria atender para que o tempo em solo das aeronaves pudesse ser limitado a apenas 30 minutos. A complexidade técnica para implementar esse requisito foi subestimada, uma vez que o algoritmo de roteamento das bagagens não era trivial. Isso levou a um atraso de 16 meses na inauguração e a um prejuízo de 1,1 milhão de dólares ao dia para a cidade de Denver (CALLEAM CONSULTING, 2008).
- Em 1996, o foguete europeu Ariane 5 explodiu em pleno ar apenas 66 segundos após ter sido lançado. A causa principal foi o requisito de reutilizar o *software* de seu antecessor, o Ariane 4, que tinha alguns campos internos com tamanhos diferentes do Ariane 5. O requisito de compatibilidade entre os modelos não foi corretamente analisado e o prejuízo estimado foi de 370 milhões de dólares (HINKEL, [201-?]).

- Em 1999, a sonda espacial climática da NASA que iria para Marte errou o ponto de inserção orbital no planeta, realizando a manobra com um erro de 100 km, o que ocasionou a destruição do equipamento e gerou um prejuízo estimado de 125 milhões de dólares. A causa foi um problema no sistema de conversão de medidas — enquanto uma equipe estava usando o sistema métrico decimal, a outra estava usando o sistema imperial (ISBELL; SAVAGE, 1999).
- Em 2019, a Boeing foi obrigada a suspender toda a frota de 387 aviões do modelo Boeing 737 Max, que voavam para 59 companhias aéreas em todo o mundo. O fato ocorreu depois que dois acidentes aéreos tiraram a vida de 346 pessoas em menos de 5 meses. A causa foi atribuída a problemas encontrados no novo *software* de controle de voo que havia sido implantado (BOEING…, 2019).

Esses são apenas alguns dos inúmeros exemplos ocorridos, cuja consequência pode ser a perda de vidas ou o prejuízo financeiro, ou ambos. No dia a dia das empresas de *software*, a consequência mais comum dos problemas em requisitos é o aumento do retrabalho, ou seja, a necessidade de refazer coisas que já estavam prontas, levando a custos não previstos inicialmente. Quanto mais longe do início do projeto ou da *sprint* se percebe o problema, mais caro é para consertar. Quanto mais atividade já tiver sido realizada sobre o requisito (especificação, implementação, testes, entrega), mais difícil e trabalhoso é consertar e maiores são os impactos negativos para o projeto ou para a *sprint*.



Saiba mais

Sprint é o termo que denomina um ciclo de desenvolvimento em uma empresa que utiliza o método ágil Scrum. Uma *sprint* tem uma duração de duas a quatro semanas e visa à entrega de um item possivelmente liberável e que agrega valor ao cliente (SCHWABER; SUTHERLAND, 2018).

Os problemas estão relacionados ao fato de que um requisito pode:

- estar incompleto;
- estar em um nível de detalhe insuficiente para as etapas seguintes do ciclo de desenvolvimento;
- conter ambiguidades ou imprecisões que levem os membros da equipe a interpretá-lo de forma diferente do que o esperado, levando a erros nas fases seguintes do ciclo de desenvolvimento;
- ser incompatível com outro requisito;
- ser tecnicamente inviável;
- ser difícil de testar e validar;
- ter priorização conflitante sob a ótica dos diversos *stakeholders*.

E por que esses problemas acontecem com os requisitos? Existem diversos motivos pelos quais esses problemas podem acontecer, e isso varia de acordo com o tipo de projeto, com o contexto de negócio, com as características da equipe e das tecnologias envolvidas. Um projeto simples, cujo contexto de negócios é dominado pela equipe técnica, vai ter menos riscos associados a requisitos do que um projeto que envolva uma equipe menos experiente ou um negócio muito inovador e sem histórico anterior.

Existem projetos que têm um cliente bem definido e usuários com perfis conhecidos. Nesse contexto, os requisitos podem ser obtidos pelo contato direto com o cliente ou com os próprios usuários. Os requisitos, nesse caso, podem falhar por problemas de comunicação entre as partes, nos quais cada uma tem uma perspectiva diferente e até mesmo um vocabulário diferente. É comum ouvir que os usuários não sabem o que querem, mas, na verdade, por mais que eles saibam, sua perspectiva pode mudar pelo simples fato de entrarem em contato com a equipe, por experimentar a primeira versão de um protótipo ou mesmo pelo fato de que suas necessidades mudam ao longo do tempo. Os efeitos da mudança vão depender de diversos fatores, mas o seu tratamento vai depender fortemente do grau de confiança entre as partes. Quanto maior o grau de confiança, mas fácil é de se gerenciar as mudanças nos requisitos e suas consequências para o projeto, para o produto e para o relacionamento entre as partes.

Há projetos que são inovadores e visam a criar um produto de *software* que não existe ainda, como é o caso das *startups* de tecnologia. Nesse cenário, de onde podem vir os requisitos? Eles podem ser resultantes da própria equipe que está criando o produto, com base no que elas enxergam que o mercado

precisa. Por esse motivo, para esses casos, um MVP (*minimum viable product* ou produto mínimo viável) é criado para analisar se a ideia pode ou não ir adiante. Quanto mais cedo surge falha, mais cedo a ideia é pivotada e um novo ciclo de desenvolvimento é iniciado, com novos requisitos.



Fique atento

Quanto maior a capacidade da equipe de perceber os requisitos que o mercado deseja para o produto, maiores são as chances de sucesso da *startup*.

Há casos ainda em que o projeto visa a uma atualização tecnológica de um produto, na qual a própria versão anterior do produto pode ser a fonte de requisitos. Nesses casos, a qualidade dos requisitos obtidos depende fortemente da qualidade do sistema anterior e da habilidade da equipe em extrair esses requisitos.

Algumas das causas mais frequentes dos problemas relacionados a requisitos, de acordo com Wiegers e Beatty (2013), são:

- os objetivos de negócio, a visão e o escopo do projeto nunca foram definidos claramente;
- os clientes estavam muito ocupados para dedicar mais tempo trabalhando com os analistas ou os desenvolvedores nos requisitos;
- o time não pode interagir diretamente com usuários representativos para entender suas necessidades;
- os clientes argumentaram que todos os requisitos eram críticos, por isso eles não estabeleceram prioridades;
- os desenvolvedores encontraram ambiguidades e informações faltantes quando foram codificar, então, eles precisaram adivinhar certas informações;
- as comunicações entre o cliente e os desenvolvedores focaram na aparência da interface do usuário e não no que os usuários necessitavam alcançar com o *software*;
- os clientes nunca aprovaram os requisitos;
- os usuários aprovaram os requisitos para um *release* ou iteração e então mudaram eles continuamente;

- o escopo do projeto aumentou à medida que as mudanças de requisitos foram aceitas, mas o cronograma se desviou porque nenhum recurso adicional foi fornecido e nenhuma funcionalidade foi removida;
- as mudanças solicitadas nos requisitos foram perdidas; ninguém sabia o *status* de solicitações de mudanças específicas;
- os clientes requisitaram certa funcionalidade e os desenvolvedores a desenvolveram, mas ninguém nunca utilizou;
- ao final do projeto, a especificação foi satisfeita, mas o usuário e os objetivos de negócio não.

Relembrando o que Frederick Brooks já nos alertava em 1987 (BROOKS, 1987, p. 13):

> A parte mais difícil ao se construir um sistema de *software* é decidir precisamente o que construir. Nenhuma outra parte do trabalho conceitual é tão difícil quanto estabelecer os requisitos técnicos detalhados, incluindo a interface com as pessoas, máquinas e outros sistemas de *software*. Nenhuma outra parte afeta tanto o sistema resultante se for feita de forma errada. Nenhuma outra parte é tão difícil de consertar depois.

2 Ciclo de vida de *software* e engenharia de requisitos

O ciclo de vida de um produto de *software* pode ser compreendido como um conjunto de etapas pelas quais o produto passa ao longo de sua existência, conforme ilustra, de forma simplificada, a Figura 1.



Figura 1. Ciclo de vida de um produto de *software*.

Concepção

Inicialmente, a necessidade de um novo produto de *software* é identificada e a viabilidade de sua construção é analisada. Como produtos de *software* podem ter diversas naturezas, a fonte dessa necessidade pode variar enormemente, especialmente hoje, quando a tecnologia permeia todas as nossas atividades, como vimos no início deste capítulo.

Na fase de **concepção**, os primeiros requisitos aparecem na forma de objetivos de negócio de alto nível, do tipo: "Precisamos de um *software* para apoiar o gerenciamento do fluxo de solicitações". Às vezes esse objetivo vem atrelado a uma meta: "(…) de modo a reduzir o tempo de prestação do serviço a 6 horas". Ainda assim, não sabemos exatamente o que precisa ser construído, até que sejam detalhados os requisitos que representam as funcionalidades do produto, conhecidos como **requisitos funcionais de produto**. Então evoluímos para algo do tipo: "O sistema deve ter uma função que emita alertas toda vez que uma solicitação estiver parada por mais de 2 horas".

Nesse momento é também comum que surjam os requisitos de projeto e de processo. Os **requisitos de projeto** podem impor restrições ao projeto em si, e podem ser relacionados com os prazos, os recursos e o orçamento. Já os **requisitos de processo** podem imprimir restrições relacionadas à forma como o projeto será desenvolvido, como, por exemplo, um método específico de desenvolvimento.

De posse dessas informações iniciais, é possível avaliar a viabilidade de se dar início ou não ao Desenvolvimento. Um projeto pode ser cancelado antes mesmo de ter sido iniciado. Isso pode ocorrer por diversos motivos, entre eles o fato de o escopo não estar suficientemente claro para que se possa comprometer recursos para a sua execução.

Desenvolvimento

A fase de **desenvolvimento** compreende todas as atividades necessárias para a produção do *software*. Isso inclui aquelas relacionadas a requisitos, análise e *design*, implementação, testes, homologação e implantação, além de todas as atividades de suporte ao desenvolvimento e à gestão do desenvolvimento, conforme ilustra a Figura 2.



Figura 2. Etapas do desenvolvimento de *software*.



Fique atento

Embora o termo "etapas do desenvolvimento" esteja sendo utilizado, aqui ele não se refere a um modelo de ciclo de vida sequencial ou cascata. Essas etapas são mais parecidas com disciplinas, que podem se intercalar e se repetir à medida que o projeto avança, de forma iterativa e incremental. A Figura 2 ilustra essas etapas.

Quando um projeto de desenvolvimento de *software* tem início, é comum que um escopo esteja estabelecido, mesmo que esse escopo ainda esteja em um nível alto de granularidade. A partir daí, as atividades relacionadas a requisitos, que tiveram início na etapa de **concepção**, são intensificadas. Os detalhes referentes à engenharia de requisitos serão explorados na próxima seção.

Dependendo do tipo e criticidade do projeto, uma vez estabelecidos os requisitos, têm início as atividades de *design*. Essa etapa consiste em projetar a solução que implementará os requisitos. Esses modelos ajudam a aprofundar a compreensão do contexto e a delinear a solução que será implementada. Mesmo empresas que utilizam métodos ágeis costumam ter uma *sprint* zero, que se dedica a conceber a arquitetura da aplicação. Nesse momento é estabelecida a arquitetura do sistema, os componentes e como eles se relacionam.

Durante a etapa de análise e *design* podem emergir novos requisitos, derivados ou não daqueles que foram inicialmente elicitados. Isso pode ocorrer porque, ao modelar a solução, o analista de sistemas pode:

- identificar requisitos técnicos adicionais, advindos da arquitetura escolhida;
- perceber que determinados requisitos não têm viabilidade técnica de execução;
- identificar requisitos conflitantes;
- identificar requisitos incompletos ou imaturos para a implementação

Uma vez definidos os componentes no nível de detalhe compatível com as necessidades do projeto, iniciam as atividades de Implementação. Note que aqui pode haver um paralelismo de atividades, pois os componentes podem ir sendo especificados ao mesmo tempo em que outros já estão sendo implementados. Nessa etapa o desenvolvedor ainda pode detectar problemas nos requisitos, devido à inviabilidade técnica de implementar.

Ao concluir a codificação, o desenvolvedor realiza os testes individuais nas unidades pelas quais foi responsável pela implementação. Nesse momento são realizadas as atividades de verificação, usando os casos de teste manuais ou automatizados.

Há empresas que utilizam a abordagem DevOps, integrando o ciclo de desenvolvimento ao ciclo de operação em produção. Isso é feito utilizando os conceitos de integração contínua (CI, *continuous integration*) e entrega contínua (CD, *continuous delivery*) e implantação contínua (CD, *continuous deployment*). Entende-se por CI o fato de haver uma integração automática à medida que o desenvolvedor sobe o seu código para o repositório e este se integra a outras partes do código que foram desenvolvidas por outros desenvolvedores. Já a entrega contínua (CD) se refere a entregar uma solução integrada para o pessoal de implantação poder realizar a passagem para o ambiente de produção, e a implantação contínua (utiliza-se a sigla CD também) se refere à passagem automática para o ambiente de produção.



Saiba mais

Quer conhecer mais sobre DevOps? Consulte o livro *Jornada DevOps: unindo cultura ágil, Lean e tecnologia para entrega de software de qualidade*, organizado por Muniz *et al.* (2019).

Manutenção

A maior parte do tempo de vida de um produto de *software* se passa na etapa de Manutenção. A partir do momento que a primeira funcionalidade entra em produção e o produto começa a ser utilizado, já começam as requisições de mudanças, sejam elas por *bugs* encontrados pelos usuários, sejam por demandas de ordem legal/fiscal ou por melhorias e novas funcionalidades.

Na etapa de Manutenção, ocorre a maior parte das atividades de gerenciamento dos requisitos. Isso significa que a rastreabilidade é a ferramenta mais importante para a análise do impacto das solicitações de mudanças.

Descontinuação

Quando o produto de *software* não atende mais aos seus propósitos ele é descontinuado, ou seja, ele é retirado do ambiente de produção. Em alguns casos, basta solicitar a desinstalação do produto, como é o caso dos aplicativos que temos em nosso celular. Mas, na maioria das vezes, a descontinuação não é uma atividade tão simples.

Para descontinuar, por exemplo, um *software* de gestão do tipo ERP (*enterprise resource planning*, ou sistema de gestão empresarial), são necessários muitos anos, pois a transição para outro produto/fornecedor não é tão simples. Muitas vezes é necessário construir outro sistema para que se possa fazer a migração de dados e processos para o novo produto. Nesses casos, é importante compreender os requisitos da desativação.



Exemplo

O negócio no qual o *software* está inserido pode ter requisitos específicos relacionados a questões de ordem legal e/ou fiscal, como o arquivamento de dados e documentos por um longo período. Por exemplo, contratos de trabalho precisam ser guardados por tempo indeterminado, enquanto outros documentos requerem prazos de 5, 10, 20 e até 30 anos. Além disso, é preciso planejar a forma como esses dados serão recuperados em caso de necessidade. Esses são requisitos que podem estar presentes no projeto de desativação de um produto de *software*.

3 Etapas da engenharia de requisitos

Agora que já entendemos quais problemas podem vir dos requisitos e compreendemos a importância da engenharia de requisitos para lidar com essas questões, vamos conhecer quais são as atividades a serem realizadas nessa etapa do desenvolvimento de *software*.

Falar em etapas da engenharia de requisitos pode sugerir que estamos nos referindo a um modelo de desenvolvimento cascata, cheio de burocracias e distante da realidade atual das empresas. Na verdade, no contexto deste capítulo, estamos considerando que as etapas da engenharia de requisitos são atividades que devem ser realizadas no desenvolvimento de produtos de *software*, independentemente do modelo de ciclo de vida que está sendo utilizado. O momento de realização dessas atividades e os artefatos envolvidos poderão variar, mas a essência é a mesma — o que varia é a forma. O objetivo é um só: construir produtos de *software* melhores e mais confiáveis!

Como você pode observar na Figura 3, a engenharia de requisitos compreende duas grandes etapas: o desenvolvimento e o gerenciamento. No desenvolvimento de requisitos estão englobadas as atividades de elicitação, análise, especificação e validação, as quais veremos em mais detalhes a seguir.



Figura 3. Etapas da engenharia de requisitos.

Elicitação de requisitos

A elicitação se refere à etapa de investigação dos requisitos. É o momento em que a equipe técnica precisa compreender o que deve ser feito. Antigamente, o termo utilizado era "levantamento de dados", no entanto, essa expressão está em desuso e, em seu lugar, usamos a expressão "elicitação de requisitos" para denotar algo mais forte, que remete a uma busca ativa pelos requisitos.

Isso significa que o analista de requisitos ou papel similar (analista de negócios, analista de sistemas, *product owner* ou equivalente) deve adotar uma postura proativa para a descoberta e o entendimento dos requisitos. Isso implica em utilizar diversas formas de elicitação, de acordo com o contexto em que o sistema está sendo desenvolvido. Outros termos associados a esta etapa podem ser: captura de requisitos, descoberta de requisitos ou aquisição de requisitos (BOURQUE; FAIRLEY, 2014).

Quando falamos da Elicitação de requisitos não estamos enfatizando que é necessário ter todos os requisitos extensivamente detalhados no início do projeto, ou que esta etapa ocorra uma única vez no início do projeto. Em projetos ágeis, os requisitos podem emergir em diversos momentos, compondo o *product backlog*. No entanto, é importante ter em mente que alguns requisitos, quando descobertos muito tardiamente no ciclo de vida de um projeto, podem implicar em grande quantidade de retrabalho e custos adicionais. Esse é o caso, por exemplo, dos requisitos que impactam fortemente sobre a arquitetura do produto — esses são mais difíceis de serem alterados posteriormente.



Fique atento

Product backlog (ou "*backlog* do produto") é o termo utilizado para denominar um conjunto de itens conhecidos que devem ser desenvolvidos para determinado produto. Quando itens do *product backlog* são priorizados para serem desenvolvidos em uma *sprint*, então, esse subconjunto é chamado de *sprint backlog*. O termo é comum em empresas que utilizam o método ágil Scrum (SCHWABER; SUTHERLAND, 2018).

O processo de elicitação exige uma intensa comunicação com os *stakeholders* do projeto. É no processo de comunicação que o analista de requisitos identifica o que realmente é imprescindível para o desenvolvimento do produto e não perde tempo detalhando o que não é importante ou que não precisa de detalhamento naquele momento. A comunicação ineficaz pode levar a requisitos incompletos e incorretos.

Há casos em que a comunicação com os *stakeholders* é direta e é possível aplicar técnicas como entrevistas e reuniões, pois existem fontes de informação humanas que compreendem o contexto de negócio e podem prover as informações necessárias para a definição dos requisitos. Delinear histórias do usuário ou mapear a jornada do usuário pode ajudar. Ambos se baseiam na narrativa de uma história de uso do sistema por um usuário.

No caso dos produtos inovadores, que estão sendo criados sem uma experiência anterior, ou que tenham um propósito disruptivo, é necessário que sejam utilizadas técnicas com o propósito de despertar a criatividade. Isto pode ser feito utilizando sessões de cocriação, como o *brainstorming* ou o *design thinking*.

Há ainda os momentos em que os requisitos são obtidos a partir de documentações como editais de licitação (no caso de contratação advinda do setor público), de normas e leis (no caso de sistemas que precisam atender a legislações específicas e órgãos regulamentadores), de documentação de outros sistemas anteriores e, até mesmo, no pior caso, do próprio código-fonte anterior.

O importante nas atividades de elicitação de requisitos é que o analista de requisitos seja capaz de compreender o contexto em que o projeto está inserido e que possa planejar de que forma irá realizar sua atividade de elicitar os requisitos. Pode ser empregando uma única técnica, pode ser empregando uma combinação de técnicas.



Fique atento

O papel do analista de requisitos é mergulhar no ambiente de negócio. O usuário não precisa de um *software*, ele precisa do serviço que o *software* presta, precisa de algo que resolva o seu problema de negócio. O mais importante é que o analista de requisitos tenha feito todos os esforços para engajar os *stakeholders* e fazer emergir os requisitos que farão do projeto um sucesso.

Análise de requisitos

A análise de requisitos é a etapa na qual nos concentramos em aprofundar o entendimento acerca dos requisitos, buscando possíveis conflitos que podem ser advindos das diferentes visões que os *stakeholders* possam ter sobre o requisito e sua prioridade no projeto de desenvolvimento. Entretanto, os problemas podem vir também de requisitos conflitantes entre si, especialmente no que se refere a **requisitos de qualidade**, também chamados de **requisitos não funcionais**. A complexidade de um requisito funcional, por exemplo, pode ser incompatível com a necessidade de desempenho exigida.

Faz parte da análise a decomposição de requisitos de alto nível em níveis apropriados de detalhe. Muitas vezes o requisito está expresso como um objetivo de negócio de alto nível e isso não é suficiente para as próximas etapas do desenvolvimento de *software*, sendo necessário decompô-lo em níveis mais detalhados.

Outro aspecto importante da análise dos requisitos diz respeito à definição dos atributos a ele associados, como o tipo do requisito, a volatilidade, o impacto sobre a arquitetura e o risco. Se um requisito, por exemplo, é volátil, ou seja, está mais sujeito a mudanças do que outros, pode ser que se decida por implementá-lo posteriormente, quando o seu entendimento esteja mais estabilizado. Da mesma forma, requisitos que tenham um grande impacto sobre a arquitetura da aplicação devem ser tratados com um cuidado redobrado, pois podem implicar em consequências para todos os demais requisitos, incluindo os requisitos de qualidade, como o desempenho da aplicação.

É importante lembrar que a extensão da análise de requisitos vai variar de acordo com a criticidade do *software* e qual o seu papel dentro do contexto no qual está inserido. Sistemas complexos, envolvendo, por exemplo *hardware* e *software*, como sistemas de *software* para a direção autônoma de veículos, precisam de uma etapa de análise de requisitos mais aprofundada, que pode até mesmo se confundir com o *design* e com as especificações arquiteturais do produto (o que envolve o sistema como um todo). Nos exemplos que vimos no início deste capítulo, pudemos observar como alguns requisitos técnicos podem levar sondas espaciais a se desintegrarem em pleno ar e aeronaves a serem retiradas de circulação após desastres aéreos provenientes do *software*. Em ambos os casos, com extensivos prejuízos materiais, mas, principalmente, com a perda de vidas humanas.

Especificação de requisitos

A especificação de requisitos é a etapa dedicada a representar os requisitos de uma forma que eles possam perdurar ao longo do tempo e possam ser verificados e validados posteriormente. Isso pode implicar em formatos diferentes de especificação que envolvem textos, diagramas e tabelas. Aqui cabe uma pequena discussão acerca do que é exatamente representar os requisitos e de que forma deveríamos realizar essa atividade.

Os métodos tradicionais de desenvolvimento, especialmente o cascata, com ciclos extensivos e exaustivos de especificação antes da codificação, já se mostraram ineficientes no passado e foram, gradualmente, substituídos. Há alguns anos que os métodos ágeis estão sendo mais amplamente adotados pela indústria, e muitos entendem que usar um método ágil significa não realizar qualquer tipo de "documentação". A palavra aqui vem entre aspas para denotar que o termo é de uso pejorativo no ambiente ágil, estando relacionado a uma ação que agrega pouco ou nenhum valor ao produto final. Agilistas mais radicais defendem que a única documentação fiel de um produto é o seu código. São dois extremos que, em geral, se antagonizam.

Vamos refletir por um instante: imagine que você estivesse no hospital e sua vida dependesse da precisão de um equipamento cirúrgico, operado, em grande parte, por um *software*. Será que você iria preferir que os projetistas tivessem documentado adequadamente os requisitos técnicos, para garantir que os envolvidos nas fases seguintes desenvolvessem o produto de forma precisa? Ou você estaria confortável em ter apenas uma história de usuário de alto nível como documentação desses requisitos?

A questão aqui é quanto de especificação é necessário antes de se dar início à implementação? A resposta é depende. Depende da complexidade do produto que está sendo desenvolvido, da criticidade de cada requisito, dos riscos de se ter um erro advindo de uma má compreensão, da experiência da equipe envolvida e até mesmo da fase em que o produto e a empresa se encontram.

É comum que empresas disruptivas de tecnologia, como as *startups*, iniciem suas atividades a partir de versões iniciais e pouco elaboradas do produto, o chamado MVP (em inglês, *minimum viable product*, isto é, produto mínimo viável). Nessa fase de sua existência, a empresa não se preocupa em especificar detalhadamente, pois seu foco é testar a viabilidade do negócio. Nesse momento a equipe é pequena, composta de duas a três pessoas, e a comunicação é direta e simples. No entanto, à medida que essa *startup* cresce e se torna uma grande empresa, não é mais possível continuar a desenvolver sem a preocupação com a especificação, pois a equipe também aumenta e a comunicação entre

os membros se torna mais complexa. Um erro no ambiente de produção pode afetar milhares de pessoas.



Fique atento

É preciso especificar os requisitos na medida certa para a situação. Nunca teremos os requisitos de forma perfeita. Precisamos ter os requisitos na forma necessária e suficiente para os objetivos do projeto e para prosseguir com as demais etapas do desenvolvimento, sejam elas denominadas de fases, sejam de etapas, iterações, estágios ou *sprints*.

Validação de requisitos

A especificação dos requisitos precisa ser validada entre os *stakeholders* e a equipe de desenvolvimento para garantir que existe uma compreensão correta e comum sobre os requisitos e que a equipe de desenvolvimento possui as condições de implementar um produto que irá satisfazer as necessidades do negócio (WIEGERS; BEATTY, 2013).

A validação pode ser realizada por meio de uma revisão sobre as especificações, que é feita por revisores designados para tal, com o apoio de um *checklist*, por exemplo. Há casos em que se conduzem *workshops* de validação, nos quais os diversos tipos de *stakeholders* são envolvidos.

Podem ainda ser usados protótipos funcionais que visam a confirmar os requisitos e até mesmo a apoiar a elicitação de novos requisitos. Nesse caso, é importante destacar a finalidade dessa validação. Existe o risco do processo de validação de um protótipo se resumir a analisar requisitos da interface com o usuário. Isso não será um problema se a interface com o usuário for um ponto crítico e que precise de muita atenção. Entretanto, não se pode perder de vista a validação das regras de negócio e dos demais tipos de requisitos, como os requisitos de qualidade.

Especificar um conjunto de casos de teste que possam ser utilizados para testar o produto também faz parte desta etapa. Esses casos de teste podem ser documentos ou casos automatizados, dependendo do nível de maturidade da organização. Testes podem ser utilizados posteriormente para verificar se o produto final cumpre o que estava na especificação (verificação) ou para validar se o produto faz aquilo que deveria fazer (validação).

Gerenciamento de requisitos

Permeando todo o ciclo dos requisitos, encontram-se as atividades relacionadas à fase de gerenciamento dos requisitos, que trata do estabelecimento de formas de rastrear os requisitos e facilitar a análise do impacto de uma solicitação de mudança para a tomada de decisão.

Pesquisa do PMI (Project Management Institute) aponta problemas no gerenciamento de requisitos (LARSON, 2014):

- Somente 49% das organizações têm recursos alocados adequadamente para o gerenciamento de requisitos.
- Somente 33% dos líderes das organizações valorizam o gerenciamento de requisitos como uma competência crítica.
- Somente 47% das organizações têm um processo formal de validação de requisitos.
- Dos recursos financeiros gastos em projetos e programas, 51% são desperdiçados devido ao gerenciamento de requisitos ineficiente.
- Dos objetivos não atendidos de projetos, 47% foram devido ao gerenciamento de requisitos ineficiente.

Para que os impactos de uma solicitação de mudança possam ser analisados de forma precisa, é necessário que se tenha conhecimento da fonte do requisito, de como os requisitos se relacionam entre si e de como eles se relacionam com os artefatos seguintes, como o código, por exemplo. Essa análise permite que tenhamos uma estimativa dos recursos e do tempo necessários para realizar a mudança solicitada, bem como para identificar quais artefatos precisam ser alterados. Isso permite que seja tomada uma decisão embasada sobre a solicitação de mudança.

Manter essa rastreabilidade entre os elementos não é uma atividade simples, especialmente em grandes produtos. Esta é uma atividade que permeia todo o ciclo de vida do produto e nasce quando os primeiros requisitos são identificados e especificados. Não é viável construir matrizes que precisem ser mantidas de forma manual, usando planilhas ou outro tipo de tabela; isso só é possível por meio de ferramentas automatizadas. Pior que não ter uma matriz de rastreabilidade é ter uma matriz desatualizada.



Referências

BOEING 737 Max Lion Air crash caused by series of failures. *BBC*, Estados Unidos, 1990. Disponível em: https://www.bbc.com/news/business-50177788. Acesso em: 30 dez. 2019.

BOURQUE, P.; FAIRLEY, R. (ed.). *SWEBOK:* guide to the *software* engineering body of knowledge version 3. Washington: IEEE Computer Society, 2014.

BROOKS, F. No silver bullet: essence and accidents of *software* engineering. *Computer*, v. 20, n. 4, p. 10–19, 1987.

CALLEAM CONSULTING. *Denver airport baggage handling system case study*. [*S. l.*], 2008. Disponível em: http://calleam.com/WTPF/wp-content/uploads/articles/DIABaggage.pdf. Acesso em: 30 dez. 2019.

HINKEL, J. N. *Ariane 5*: um erro numérico (overflow) levou à falha no primeiro lançamento. São José dos Campos, [201-?]. Disponível em: https://www.ime.uerj.br/~demoura/Especializ/Ariane/. Acesso em: 30 dez. 2019.

IEEE. *IEEE Std 610.12-1990*: IEEE Standard Glossary of *Software* Engineering Terminology. Los Alamitos: IEEE Computer Society Press, 1990.

ISBELL, D.; SAVAGE, D. Mars climate orbiter failure board releases report, numerous NASA actions underway in response. *In:* MARS POLAR LANDDER. Washington, 1999. Disponível em: https://mars.jpl.nasa.gov/msp98/news/mco991110.html. Acesso em: 30 dez. 2019.

LARSON, E. I still don"t have time to manage requirements: my project is later than ever. *In*: PMI® GLOBAL CONGRESS, 2014, Phoenix, AZ. *Proceedings*... Newtown Square, PA: Project Management Institute, 2014. Disponível em: https://www.pmi.org/learning/library/poor-requirements-management-source-failed-projects-9341. Acesso em: 30 dez. 2019.

MUNIZ, A. *et al. Jornada DevOps*: unindo cultura ágil, Lean e tecnologia para entrega de *software* de qualidade. Rio de Janeiro: Brasport, 2019.

SCHWABER, K.; SUTHERLAND, J. *Guia do SCRUM:* um guia definitivo para o SCRUM: as regras do jogo. [*S. l.*], 2018. Disponível em: https://www.scrumguides.org/index.html. Acesso em: 30 dez. 2019.

WIEGERS, K. E.; BEATTY, J. *Software requirements*. 3. ed. Redmond: Microsoft Press, 2013.

Leitura recomendada

PRESSMAN, R.; MAXIM, B. *Engenharia de software*: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.



INOVAÇÃO
E TECNOLOGIA

unidade
1

Parte 2

Requisitos de Software

O conteúdo deste livro é disponibilizado por SAGAH.

SOLUÇÕES EDUCACIONAIS INTEGRADAS

Requisitos de *software*

Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Classificar os requisitos de um projeto de *software*.
- Priorizar os requisitos em um projeto de *software*.
- Aplicar critérios de qualidade para avaliar a qualidade dos requisitos de *software*.

Introdução

Produtos de *software* fazem parte do nosso dia a dia em praticamente todas as atividades que realizamos, desde as mais simples até as mais sofisticadas. Empresas de todos os setores também dependem fortemente de produtos de *software*, que são utilizados tanto para a realização de atividades produtivas quanto para as de controle e gestão. Quando um *software* falha, seus transtornos são sentidos imediatamente e impactam na forma como as atividades cotidianas de empresas e indivíduos são realizadas.

Uma das etapas do ciclo de desenvolvimento de *software* que tem o maior potencial de causar danos futuros ao projeto e ao produto final em si é a engenharia de requisitos. Quando não compreendemos adequadamente o que deve ser construído, falhamos para estimar prazos e recursos, gerando prejuízos financeiros para o projeto. Mais importante do que isso, uma falha na compreensão dos requisitos pode se propagar para o restante das atividades do desenvolvimento, gerando produtos que não satisfazem as necessidades e que podem, inclusive, gerar danos físicos ou levar à perda de vidas.

Sabemos, no entanto, que os recursos para o desenvolvimento não são infinitos e a velocidade do mercado exige que novos produtos ou funcionalidades sejam lançados cada vez mais rápido. Então como fazer para equilibrar o tempo disponível para o desenvolvimento com a qualidade exigida do produto final? O primeiro passo é garantir bons requisitos, de modo que a agilidade possa ser acompanhada de qualidade.

Neste capítulo você vai estudar os diferentes tipos de requisito e quais os desdobramentos dessas categorias. Verá também como os requisitos podem ser priorizados em função de suas características e das características do contexto. Por fim, vai ler sobre como aplicar critérios para analisar a qualidade dos requisitos, para que possam contribuir de forma efetiva para a qualidade das próximas etapas do desenvolvimento de *software*.

1 Tipos de requisitos de *software*

Inicialmente, precisamos definir o que é requisito. Inúmeras definições podem ser encontradas e optamos, neste livro, por aderir ao *Glossário padrão de terminologia de engenharia de software* do *Institute of Electrical and Electronic Engineers* (IEEE, 1990) que define requisito como:

1. Uma condição ou capacidade necessária para um usuário resolver um problema ou alcançar um objetivo.
2. Uma condição ou capacidade que deve ser atendida ou tida por um sistema ou componente do sistema para satisfazer a um contrato, padrão, especificação ou outro documento formalmente imposto.
3. Uma representação documentada de uma condição ou capacidade conforme estabelecido em 1 e 2.

Ao olhar mais detidamente os termos envolvidos nessa definição, você percebe que os principais aspectos dos requisitos estão cobertos: foco no problema que a solução deve resolver; foco na satisfação a uma imposição (que pode, por exemplo, ser uma legislação); e, finalmente, foco na representação documentada. Note ainda que essa definição não direciona para um método específico de desenvolvimento, uma vez que não especifica "como", mas sim "o quê".

Outra forma de entender o que é requisito é se apoiar na definição de Sommerville e Sawyer (1997), que estabelecem que requisitos são a especificação do que precisa ser implementado, são descrições de como o sistema deve se comportar, ou são uma propriedade ou atributo do sistema. Os requisitos podem, por exemplo, ser uma restrição no processo de desenvolvimento do sistema. Aqui temos o conceito de requisito do processo, que será tratado posteriormente neste capítulo.

O termo **requisito** tem diversas interpretações diferentes na engenharia de *software*. Cada autor apresenta uma forma diferente de classificar os requisitos. Pohl e Rupp (2015) classificam os requisitos de acordo com as definições do Quadro 1.

Quadro 1. Tipos de requisitos de acordo com Pohl e Rupp (2015)

Tipo do requisito	Definição
Requisito funcional	É um requisito que se refere a um resultado de comportamento que deve ser provido por uma funcionalidade do sistema.
Requisito de qualidade	É um requisito que pertence a uma preocupação com a qualidade que não é coberta pelos requisitos funcionais.
Restrição	É um requisito que limita o espaço da solução além do que é necessário para atender aos requisitos funcionais e requisitos de qualidade.

Fonte: Adaptado de Pohl e Rupp (2015).

Um **requisito funcional** é, como o próprio nome diz, uma funcionalidade requerida pelos *stakeholders* para cumprir algum objetivo de negócio. Representa o que os desenvolvedores deverão implementar para que os usuários possam realizar as suas atividades. Geralmente, os requisitos funcionais são expressos em frases do tipo "o sistema deve". Por exemplo: "O sistema deve permitir que o usuário pague com cartão de débito ou crédito".

Os **requisitos de qualidade** se referem a como o *software* vai operar sob determinadas circunstâncias e, geralmente, estão relacionados a questões como desempenho, disponibilidade, usabilidade, portabilidade, escalabilidade etc. Eles também são chamados de **requisitos não funcionais**.

É comum que os requisitos não funcionais sejam negligenciados no início do projeto — e aí reside um grande problema, pois eles, geralmente, têm forte impacto sobre a arquitetura da aplicação, e são os requisitos mais difíceis de se retificar posteriormente. Para conseguirmos atender a um requisito específico de desempenho, por exemplo, é necessário fazer escolhas arquiteturais que darão suporte a esse requisito. Se isso não for planejado em tempo de projeto (*design*), será muito difícil corrigir depois que a implementação estiver avançada.

As **restrições** são consideradas como os requisitos sobre os quais a equipe de desenvolvimento não tem gestão. Eles não são implementados no produto de *software*, mas influenciam a forma como o produto será implementado. Geralmente, relacionam-se com as escolhas sobre tecnologias (*hardware*, ambientes, linguagens de programação etc.) e processos de desenvolvimento (ciclo de vida, métodos, procedimentos de gestão etc.), além de restrições do próprio projeto que gera o produto (prazos, custos, recursos etc.). As restrições são também comumente chamadas de **requisitos de processo** e **requisitos de projeto**.

Wiegers e Beatty (2013) utilizam uma visão um pouco diferente, incluindo, além dos tipos em si, o nível de abstração dos requisitos, conforme você pode ver no Quadro 2.

Quadro 2. Tipos de requisitos de acordo com Wiegers e Beatty (2013)

Termo	Definição
Requisito de negócio	Um objetivo de alto nível de negócio da organização que constrói um produto ou de um cliente que o adquire.
Regra de negócio	Uma política, um guia, um padrão, uma regulamentação que define ou restringe algum aspecto de negócio. Não é um requisito do *software* em si, mas a origem de diversos tipos de requisitos de *software*.
Restrição	Uma restrição que é imposta sobre as escolhas disponíveis para o desenvolvedor para o projeto (*design*) e construção de um produto.
Requisito de interface externa	Uma descrição de uma conexão entre um sistema de *software* e um usuário, outro sistema de *software* ou um dispositivo de *hardware*.
Característica (*feature*)	Uma ou mais capacidades do sistema logicamente relacionadas que fornecem valor para um usuário e são descritas por um conjunto de requisitos funcionais.
Requisito funcional	Uma descrição de um comportamento que o sistema deve exibir sob condições específicas.
Requisito não funcional	Uma descrição de uma propriedade ou característica que o sistema deve exibir ou uma restrição que ele deve respeitar.

(Continua)

(Continuação)

Quadro 2. Tipos de requisitos de acordo com Wiegers e Beatty (2013)

Termo	Definição
Atributo de qualidade	Um tipo de requisito não funcional que descreve um serviço ou característica de desempenho de um produto.
Requisito de sistema	Um requisito de alto nível para um produto que contém múltiplos subsistemas, que pode ser todo de *software* ou de *software* e *hardware*.
Requisito de usuário	Um objetivo ou tarefa que classes específicas de usuários devem ser capazes de executar com um sistema ou um atributo de produto desejado.

Fonte: Adaptado de Wiegers e Beatty (2013).

De acordo com a visão de Wiegers e Beatty (2013), os **requisitos de negócio** estão mais diretamente ligados aos motivos pelos quais um *software* está sendo desenvolvido, ou seja, quais são os objetivos de negócio que se deseja atingir e qual o valor que esse produto agregará ao negócio. Geralmente, são providos pelo patrocinador, executivos da organização ou pelo pessoal de produto (gerência de *marketing*, por exemplo). Os requisitos de negócio são como grandes objetivos que todos precisam estar de acordo, pois serão a base da maioria das decisões futuras sobre o projeto e até mesmo a priorização das entregas. Com base nos requisitos de negócio também serão avaliadas as solicitações de mudanças que certamente surgirão ao longo do desenvolvimento.

Embora pareça bastante óbvio que um projeto deva ser iniciado de forma alinhada aos objetivos de negócio, nem sempre isso acontece, pois, muitas vezes, a visão sobre esse relacionamento não é muito clara. Como consequência, a equipe técnica pode ter que lidar com visões conflitantes entre os diversos *stakeholders*, o que pode ter impacto nas atividades de desenvolvimento e no resultado final. Outro efeito colateral possível é que a equipe tente abranger as visões de todos os possíveis *stakeholders*, deixando de focar naqueles que são mais relevantes, levando o produto a abranger um escopo muito maior do que o inicialmente previsto. Discutiremos mais esse tópico nas próximas seções.



Exemplo

Exemplos de benefícios quantificáveis financeiramente sob a ótica do negócio:

- incrementar as vendas em 25% em 6 meses;
- aumentar a quantidade de engajamentos na página da marca em 15% nos próximos 3 meses;
- reduzir os gastos com vendedores presenciais em 40% até o final do próximo ano.

Exemplos de benefícios não financeiros podem incluir:

- aumentar o índice de satisfação dos clientes em 10% no próximo trimestre;
- receber no máximo 10 chamados por mês no *service desk* após 3 meses da implantação do produto.

Como se pode observar nos quadros anteriores, existem formas diferentes de classificar os requisitos, mas, basicamente, podemos considerar que existem requisitos que são do **produto** de *software* em si, ou seja, que serão construídos pela equipe de desenvolvimento e farão parte do produto entregue; e existem requisitos que não são do produto, ou seja, fazem parte do **projeto** ou do **processo**. Os requisitos que são do produto, podem ser **funcionais** (o que o *software* deve fazer) ou **não funcionais** (como o *software* deve fazer). Os requisitos de projeto e de processo são sempre não funcionais.

Outro conceito importante é o de **requisito de sistema**. Embora, na maioria das empresas, o termo **sistema** seja usado para designar um produto de *software*, seu conceito é mais amplo do que isso. Entende-se por sistema, a composição de *hardware*, *software* e processos. Portanto, requisitos de sistema são requisitos de mais alto nível, que devem ser atendidos pelo produto como um todo, e o requisito de *software* se refere ao requisito do sistema que é alocado ao componente de *software*. Isso se torna particularmente importante quando falamos de sistemas ciberfísicos (CPS, *cyber-phisical systems*), cuja composição envolve a parte física (ambiente, sensores, atuadores) e a parte ciber (*software*). Esse é o caso, por exemplo, das funcionalidades de *park assist* (estacionamento automático) nos automóveis de luxo. Um requisito de sistema poderia ser: "O sistema deve buscar uma vaga para estacionar o veículo". Esse requisito do sistema envolve elementos físicos (sensores para detectar espaços vazios) e elementos de *software* (para identificar se o carro cabe naquele espaço).



Saiba mais

As normas da ISO oferecem um bom suporte para todos os processos relacionados a requisitos:

- Para mais detalhes sobre processos relacionados a requisitos de sistema, uma boa dica é olhar a norma internacional ISO/IEC 15288:2015, *Systems and Software Engineering — Systems Lyfe Cycle Processes*. Nela é possível identificar o processo de definição de requisitos de sistema, que estabelece as atividades e os resultados esperados quando se executa o processo.
- Para mais detalhes sobre processos relacionados aos requisitos de *software*, você pode usar a norma internacional ISO/IEC/IEEE 12207:2017, *Systems and Software Engineering — Software Lyfe Cycle Processes*. Procure pelos processos de definição de requisitos de sistema/*software*. Embora essa norma seja dedicada aos processos de *software*, ela trata os requisitos de forma mais ampla, incluindo requisitos de sistema.
- Para mais detalhes sobre requisitos de forma mais ampla, consulte a norma internacional ISO/IEC 29148:2018, *Systems and Software Engineering — Lyfe Cycle Processes — Requirements Engineering*. Ela inclui mais detalhes sobre como implementar os processos que estão descritos nas duas normas anteriores.

É importante compreendermos adequadamente esses diferentes tipos de requisitos, pois cada um deles será desdobrado em um artefato diferente do ciclo de vida. Os requisitos relacionados ao **produto** farão parte de um documento de especificação de requisitos (ou artefato similar, como cartões com histórias de usuário nos métodos ágeis), enquanto os requisitos de **processo** e de **projeto** serão parte do plano de projeto (ou artefato que tenha função similar). Já requisitos de **sistema** se desdobrarão em requisitos relacionados à parte física (*hardware*) e requisitos relacionados à parte de *software*, que podem estar em um mesmo artefato, mas que se desdobrarão em implementações distintas.

2 Priorização de requisitos

Uma das maiores causas de fracassos em projetos está relacionada ao seu tamanho. Quanto maior o projeto, maiores são as chances de ele não atingir suas metas de prazo, custo, escopo e qualidade. Uma forma de minimizar esses efeitos é dividi-lo em partes menores e mais facilmente gerenciáveis. Esse é o princípio que foi adotado pelos modelos de ciclo de vida iterativos e incrementais e, mais recentemente, pelas metodologias ágeis, que preveem a divisão de trabalho em *sprints* ou iterações, com durações curtas, de,

no máximo, um mês. Dessa forma, os requisitos podem ser conhecidos logo no início, mas eles serão detalhados à medida que forem sendo priorizados para a alocação em uma entrega.

A forma de priorizar os requisitos vai depender do contexto em que o projeto está inserido e das próprias características do produto e da tecnologia envolvida. Alguns dos fatores que podem ser usados na priorização envolvem, mas não estão restritos a:

- **Potencial de valor a ser agregado ao negócio pelo requisito:** requisitos que agregam mais valor ao negócio geralmente são alocados às primeiras entregas, enquanto requisitos de menor valor são alocados a versões posteriores.
- **Dependência do requisito em relação a outros requisitos:** muitas vezes um requisito não pode ser implementado isoladamente, requerendo que sejam priorizados outros requisitos que dão suporte a ele, na mesma versão do produto.
- **Análise de requisitos implícitos que podem estar invisíveis:** no momento da priorização de requisitos, pode ser que requisitos implícitos que estavam invisíveis em um primeiro momento sejam identificados e precisem ser priorizados juntamente com o requisito principal. Isso é muito comum, por exemplo, nos requisitos de segurança de acesso, que podem implicar em uma série de requisitos de apoio (que podem até mesmo virar um subsistema).
- **Experiência com a tecnologia por parte da equipe do projeto:** a falta de experiência da equipe de desenvolvimento com a tecnologia pode levar à priorização de requisitos que não sejam exatamente os que geram mais valor ao cliente, mas sim aqueles que possibilitam o aprendizado mais rápido e com menor risco para o projeto como um todo.
- **Experiência no domínio de aplicação por parte da equipe:** analogamente ao anterior, a falta de experiência da equipe com o domínio de aplicação requer cuidados redobrados na priorização, especialmente, quando houver o risco de a equipe subestimar a complexidade de um requisito em função de sua falta de experiência com o domínio de negócio. Nesses casos, a proximidade com o usuário, cliente ou seus representantes é fundamental.
- **Relacionamentos com outros sistemas (*hardware* ou *software*):** quando o *software* implicar em relacionamentos com outros *softwares* ou com *hardware*, na forma de um sistema, é necessário que as dependências

entre os requisitos sejam claramente explicitadas, de modo que o desenvolvimento de requisitos dependentes possa ser realizado de forma síncrona e não isolada.

- **Demandas de implementações para atender a determinações de ordem legal ou regulatória:** é comum que demandas de ordem legal ou que sejam demandadas por órgãos regulamentadores tenham datas específicas para entrar em vigor. Requisitos que implementam essas necessidades precisam ser priorizados em versões compatíveis com as datas demandadas.
- **Requisitos que não possuem alternativas manuais aceitáveis:** há casos em que o procedimento manual para a realização da atividade de negócio é oneroso e sujeito a falhas. Nesses casos, requisitos que substituem esses processos são priorizados em detrimento de outros, cujos procedimentos manuais sejam menos onerosos ou representem menos riscos ao negócio.

A priorização pode ser feita por um único representante do usuário, como um *product owner* (PO) do método Scrum, ou por um grupo de *stakeholders*. Isso pode acontecer virtualmente, usando ferramentas de colaboração, ou em *workshops* de priorização presenciais. A forma vai depender da quantidade de *stakeholders* envolvidos e do nível de conflito que possa haver entre os requisitos de produto a serem priorizados e as restrições do projeto/processo. Quanto mais críticos forem esses conflitos, maior a necessidade de se utilizar procedimentos colaborativos de decisão.

3 Critérios de qualidade de um bom requisito

Não importa o modelo de ciclo de vida ou o processo de desenvolvimento utilizado, um requisito mal compreendido e mal documentado será sempre um problema para o projeto que envolve *software*. Por esse motivo, é imprescindível que os requisitos, independentemente de sua categoria, estejam claramente identificados, de modo que as atividades que dependem deles possam ser realizadas com sucesso. Como podemos estimar adequadamente o esforço para a implementação de um requisito se sua descrição está ambígua e imprecisa? Como evitar o retrabalho no desenvolvimento se o requisito não foi detalhado de forma suficiente? Como prever os testes necessários se não houver detalhes para isso?

Primeiramente, é importante não confundir um requisito bem especificado com uma etapa de requisitos infinita, que se perde nos mínimos detalhes e que não avança na entrega de valor para o cliente. Aqui é ressaltada a importância de termos o requisito especificado de forma clara e em nível de detalhe compatível com o contexto. Por **contexto** entendemos as variáveis que podem afetar essa decisão, como experiência da equipe de desenvolvimento com a tecnologia e com a área de negócio, criticidade do produto em desenvolvimento, impacto dos possíveis erros que possam ser derivados (*software* de missão crítica, por exemplo) e nível de envolvimento dos *stakeholders*.



Fique atento

É comum achar que, ao utilizar métodos ágeis, não é necessário gastar muito tempo com os requisitos, pois eles vão, invariavelmente, mudar. Na verdade, temos sim que dispender todo o esforço necessário para termos certeza de que compreendemos adequadamente cada demanda que vai para desenvolvimento, do contrário contribuiremos, entre outras coisas, para o aumento da insatisfação do cliente com o produto entregue.

O que acontece em métodos ágeis como o Scrum é que o *product backlog* tem itens em níveis diferentes de maturidade. Os itens que já estão sendo priorizados para serem produzidos na próxima *sprint* devem estar maduros o suficiente para serem estimados, alocados à *sprint* e implementados. Haverá, por outro lado, itens menos maduros no *product backlog*, que serão detalhados no momento oportuno, conforme a sua prioridade.

De acordo com Schwaber e Sutherland (2018), itens de alta prioridade no *product backlog* costumam ser mais claros e detalhados do que itens de prioridade mais baixa. Estimativas mais precisas são feitas com base na maior clareza e no maior nível de detalhe; quanto menos prioritário, menos detalhado. Note que os criadores do Scrum ressaltam que os itens menos prioritários estarão menos detalhados, mas no momento em que se tornarem prioritários, terão que estar claros e detalhados.

De acordo com a Norma Internacional ISO/IEC/IEEE 29148 (ISO/IEC/IEEE, 2018), um requisito bem especificado contém um ou mais destes itens:

- deve ser atendido ou tido por um sistema para resolver um problema, atingir um objetivo ou tratar de uma preocupação de um *stakeholder*;
- é qualificado por condições mensuráveis;

- é limitado por restrições;
- define o desempenho do sistema quando usado por um *stakeholder* específico ou é a capacidade correspondente de um sistema, mas não a capacidade de um usuário, operador ou outro *stakeholder;*
- pode ser verificado (isto é, a realização de um requisito no sistema pode ser demonstrada).

Como documentar um requisito

Diversas são as formas de documentar um requisito, incluindo linguagem natural, tabelas, planilhas, diagramas, vídeos, áudios, fotos etc. Neste capítulo, vamos nos concentrar na forma escrita, usando a linguagem natural. Ela pode ser utilizada tanto para os requisitos funcionais quanto para os não funcionais, sejam eles de produto, sejam de projeto ou processo. Serve também para especificar os requisitos ou objetivos de negócio, que vão nortear todo o desenvolvimento. As demais formas podem funcionar como um apoio ou complemento ao que está descrito em linguagem natural.

A primeira preocupação é que a linguagem natural apresenta ambiguidades que podem comprometer a precisão dos requisitos e a sua compreensão pelo público-alvo. Escrever requisitos não é como escrever qualquer outro tipo de texto, de ficção ou não. Não há uma forma de ensinar a escrever um bom requisito, pois isso vem com a experiência, mas, algumas dicas podem ajudar a começar:

- **Ótica da escrita:** lembre-se que um requisito tem que ser visto sob a ótica de quem lê o documento, e não sob a ótica de quem escreve. Muitas vezes o requisito está completamente claro para quem escreveu, mas gera dúvidas para quem lê. Nesse caso, ele precisa ser revisado.
- **Tamanho das frases:** frases longas dificultam a leitura e podem levar a interpretações incorretas. Dê preferência a frases curtas e diretas. Releia algumas vezes e elimine qualquer palavra que não agregue compreensão ao conteúdo.
- **Correção na escrita:** os requisitos constituem um documento técnico e devem, portanto, seguir as normas da língua portuguesa escrita. Isso quer dizer que devem estar corretos gramaticalmente e sem erros de digitação. Inclua aí uma preocupação extra com a pontuação (que pode modificar completamente o sentido da frase).

- **Uso da voz ativa:** dê preferência para frases usando a voz ativa (por exemplo, "o sistema deve prover uma função de cálculo do imposto") em vez da voz passiva (por exemplo "uma função de cálculo do imposto deve ser provida"). Misturar voz passiva e ativa também torna o texto difícil de ler.
- **Uso da forma positiva:** também é preferível usar a forma positiva e evitar a negativa do tipo "não deve". Alguns chamam essa forma de requisito inverso, mas ela deve ser evitada. Frases com muitas negativas também geram complexidade no entendimento. Tudo o que precisamos ler duas vezes para entender não está bem escrito.
- **Termos ambíguos:** há termos que, naturalmente, levam a ambiguidades e indeterminações, e devem ser removidos de qualquer especificação de requisitos, como alguns, muitos, poucos, melhor, pior, robusto, adequado, rápido, amigável. Também há expressões que dificultam a precisão, como quando apropriado, quando possível, quando aplicável, se possível, se necessário.
- **Tempo verbal:** o tempo verbal empregado pode trazer um significado adicional implícito ao requisito, e isso nem sempre é desejável. Por exemplo: "o sistema deve" sugere que é um requisito obrigatório, mas o "sistema deveria" sugere uma interpretação de que o sistema poderia não fazer, implicando em um requisito menos prioritário. Questões de prioridade não devem ser tratadas pelo tempo verbal, mas sim por atributos de prioridade específicos, uma vez que podem mudar ao longo do tempo.
- **Acrônimos e jargões:** acrônimos (siglas) e jargões também constituem um ponto frágil na especificação de requisitos. O recomendado é manter um glossário compartilhado entre todos os membros do projeto (internos e externos), de modo que não sejam usados termos de forma inconsistente.
- **O que e não como:** lembre-se de que, idealmente, os requisitos devem definir o que deve ser feito, e não como deve ser implementado.



Saiba mais

As normas internacionais da ISO (International Organization for Standardization), no Brasil editadas pela ABNT (Associação Brasileira de Normas Técnicas), fazem uma clara distinção entre os itens que são escritos com o tempo verbal **deve** (*shall*) e com o tempo verbal **deveria** (*should*). As cláusulas de uma norma que possuem o verbo **deve** são obrigatórias e, no caso de normas certificadoras, serão requeridas pelo auditor como parte do processo de auditoria para concessão do certificado. Já as cláusulas que estão descritas com o verbo **deveria** não são exigidas, e a empresa pode opcionalmente não implementá-las (ISO/IEC/IEEE, 2018).

Uma forma para a escrita dos requisitos é fazê-la em três etapas:

1. Escrever a primeira versão.
2. Realizar a leitura crítica dessa versão se colocando no lugar do leitor.
3. Solicitar a revisão de um colega, a chamada **revisão por pares**.

Com o tempo, a escrita vai se tornando mais natural e os primeiros erros não serão mais cometidos, especialmente se eles, anteriormente, levaram a alguma consequência indesejada, como o retrabalho de implementação por ambiguidade nos requisitos. Leve a sério os *feedbacks* recebidos dos seus pares e elabore seu próprio *checklist* para autorrevisão dos requisitos — isso vai ajudá-lo a aprimorar essa habilidade ao longo do tempo.

Características de um bom requisito

Alguns critérios de qualidade que podem ser utilizados para nortear a escrita e a verificação dos requisitos estão listados a seguir, baseados nas recomendações de Wiegers e Beatty (2013) e da Norma Internacional ISO/IEC/IEEE 29148 (ISO/IEC/IEEE, 2018). Eles podem ser organizados no formato de um *checklist*, a ser aplicado para analisar cada requisito antes de sua liberação para a equipe de implementação, ou mesmo antes de liberá-los para a revisão por pares. O Quadro 3 apresenta as características de um bom requisito.

Quadro 3. Características de um bom requisito

Atributo	Definição
Completo	O requisito está especificado de forma completa e possibilita que o desenvolvedor o implemente.
Correto	O requisito reflete o que o usuário, cliente ou seus representantes desejam.
Único	O requisito descreve uma única capacidade, característica, restrição ou atributo de qualidade.
Viável	O requisito é viável técnica e financeiramente para ser implementado, de acordo com as restrições do projeto.
Necessário	O requisito tem um motivo de existir, que é representado pelo seu relacionamento com uma fonte de informação e com um objetivo de negócio.
Priorizado	O requisito tem uma prioridade atribuída para que possa ser alocado a uma versão do *software*.
Não ambíguo	O requisito não contém ambiguidades que levem os *stakeholders* a interpretá-lo de forma diferente.
Verificável	O requisito pode ser verificado posteriormente à sua implementação.
Conforme	O requisito está em conformidade com os padrões de especificação estabelecidos, se houver.

Fonte: Adaptado de Wiegers e Beatty (2013) e ISO/IEC/IEEE (2018).

Características de um bom conjunto de requisitos

Como você pode observar no quadro anterior, cada requisito, individualmente, deveria atender às características listadas para ser considerado um bom requisito, apto a servir de base para as atividades seguintes do ciclo de desenvolvimento. Adicionalmente, existem características que precisam ser consideradas para o conjunto de requisitos de modo que a solução possa entregar o valor esperado pelos *stakeholders*, respeitando as suas restrições. Entende-se por **solução** não necessariamente o produto completo, mas uma versão entregue do produto.

No Quadro 4, compilado a partir das definições de Wiegers e Beatty (2013) e da Norma Internacional ISO/IEC/IEEE 29148 (ISO/IEC/IEEE, 2018), podemos observar as características de um bom conjunto de requisitos. Diferentemente do quadro anterior, vemos que aqui aparecem atributos que são analisados para um ou mais requisitos. Por exemplo, analisar se o conjunto de requisitos está completo se refere a buscar pelos requisitos implícitos ou invisíveis, por exemplo. Pode ser que determinado requisito só possa ser implementado com a preparação de algum tipo de infraestrutura, como a existência de cadastros prévios, por exemplo. Muitas vezes, esses cadastros não são explicitamente identificados pelos *stakeholders*, por acharem que já estavam subentendidos.

Outro aspecto importante é a viabilidade de um requisito, quando analisado em conjunto com os demais. Podemos ter, por exemplo, um requisito de desempenho (tempo de resposta) que seja incompatível com um requisito de processo (utilizar os equipamentos disponíveis atualmente).

Quadro 4. Características de um bom conjunto de requisitos

Atributo	Definição
Completo	O conjunto de requisitos está completo.
Consistente	Os requisitos são consistentes entre si e com os requisitos de mais alto nível dos quais são originários, não apresentando conflitos.
Modificável	Cada requisito tem uma identificação única que permite que ele seja modificado quando necessário.
Compreensível	O conjunto de requisitos está escrito de forma que é possível identificar claramente o que é esperado do *software* no ambiente do qual ele faz parte.
Viável	O conjunto de requisitos pode ser executado dentro das restrições estabelecidas (técnicas, de custo e prazo), dentro de riscos aceitáveis.
Capaz de ser validado	O conjunto de requisitos pode ser validado quanto às necessidades dos *stakeholders* dentro das restrições (como custo, prazo, técnica, conformidade com regulamentações e legislações).
Rastreável	O conjunto de requisitos pode ser rastreado às suas origens (*backward*) e também aos demais elementos, como requisitos derivados, elementos de *design*, código e casos de teste (*forward*).

Fonte: Adaptado de Wiegers e Beatty (2013) e ISO/IEC/IEEE (2018).

Nível de detalhamento dos requisitos

Uma pergunta difícil de responder é sobre quão detalhados devem ser os requisitos. Isso vai depender de alguns fatores relacionados ao produto de *software* em si e ao contexto em que ele está sendo desenvolvido, conforme você pode acompanhar a seguir (WIEGERS; BEATLY, 2013).

- **Mais detalhes:**
  - o trabalho está sendo executado para um cliente externo;
  - o desenvolvimento e/ou o teste serão terceirizados;
  - os membros do projeto estão geograficamente dispersos;
  - os testes de sistema serão realizados com base nos requisitos;
  - estimativas precisas são necessárias;
  - a rastreabilidade entre os requisitos é necessária.
- **Menos detalhes:**
  - o trabalho está sendo realizado internamente para a sua empresa;
  - os clientes estão bastante envolvidos;
  - os desenvolvedores possuem uma grande experiência no domínio;
  - existem precedentes, como no caso em que uma aplicação está sendo substituída;
  - um pacote de solução será utilizado.

Em resumo, se a equipe é experiente no domínio do negócio e os clientes estão próximos e acessíveis, o nível de detalhamento dos requisitos pode ser menor, pois os riscos envolvidos também serão menores. No entanto, quando a equipe não é tão experiente no domínio do negócio, os riscos são maiores e, portanto, um nível maior de detalhamento dos requisitos é necessário.



Referências

IEEE. *IEEE Std 610.12-1990*: IEEE Standard Glossary of Software Engineering Terminology. Los Alamitos: IEEE Computer Society Press, 1990.

ISO/IEC/IEEE. *ISO/IEC/IEEE 12207:2017*: systems and software engineering: software life cycle processes. Switzerland: ISO, 2017.

ISO/IEC/IEEE. *ISO/IEC/IEEE 15288:2015*: systems and software engineering: systems life cycle processes. Switzerland: ISO, 2015.

ISO/IEC/IEEE. *ISO/IEC/IEEE 29148:2018*: systems and software engineering: life cycle processes: requirements engineering. Switzerland: ISO, 2018.

POHL, K.; RUPP, C. *Requirements engineering fundamentals*: a study guide for the certified professional for requirements engineering exam, foundation level, IREB Compliant. 2. ed. Santa Barbara: Rock Nook, 2015.

SCHWABER, K.; SUTHERLAND, J. *Guia do SCRUM:* um guia definitivo para o SCRUM: as regras do jogo. [*S. l.*], 2018. Disponível em: https://www.scrumguides.org/index.html. Acesso em: 30 dez. 2019.

SOMMERVILLE, I.; SAWYER, P. *Requirements engineering*: a good practice guide. Chichester: John Wiley & Sons, 1997.

WIEGERS, K. E.; BEATTY, J. *Software requirements*. 3. ed. Redmond: Microsoft Press, 2013.

Leituras recomendadas

BOURQUE, P.; FAIRLEY, R. (ed.). *SWEBOK:* guide to the *software* engineering body of knowledge version 3. Washington: IEEE Computer Society, 2014.

PRESSMAN, R.; MAXIM, B. *Engenharia de software*: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

WIEGERS, K. *More about software requirements*: thorny issues and practical advices. Redmond: Microsoft Press, 2006.

Parte 3

Seleção de Técnicas de Elicitação de Requisitos de Software

O conteúdo deste livro é disponibilizado por SAGAH.

SAGAH SOLUÇÕES EDUCACIONAIS INTEGRADAS

Seleção de técnicas de elicitação de requisitos de *software*

Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Identificar fontes de informação de requisitos de projetos de *software*.
- Reconhecer as características das técnicas de elicitação de requisitos de projetos de *software*.
- Selecionar a técnica de elicitação de requisitos de acordo com as características do projeto.

Introdução

À medida que a tecnologia faz cada vez mais parte de nossas atividades cotidianas, cresce nossa dependência do correto funcionamento de dispositivos e *softwares* neles contidos. Utilizamos nossos *smartphones* para apoiar a maioria das nossas atividades pessoais e profissionais, desde consultar como está o trânsito no caminho para a faculdade ou o trabalho até checar a temperatura para escolher a roupa adequada ou consultar a conta para ver se o pagamento entrou. Não admitimos mais que os bancos não tenham todos os serviços *on-line* ou que tenhamos que nos deslocar até a pizzaria para buscar o lanche. Temos tudo na ponta dos dedos!

Para que essas facilidades nos alcancem na forma de funcionalidades nos aplicativos do celular ou nos *sites* na internet, muita coisa precisa acontecer antes. São necessários profissionais com diversas habilidades e competências, esforço conjunto e horas de desenvolvimento e testes. Em algum momento, pessoas criaram esses serviços.

De onde saem as ideias para essas funcionalidades? Quem define o que um sistema deve executar e como deve executar? Como se faz

para obter essas definições? A resposta é: depende! Depende do tipo de *software*, do contexto em que o projeto está sendo desenvolvido, da tecnologia e dos recursos disponíveis. Há casos em que é possível conversar com alguém responsável por essas definições e há casos em que é a lei que determina o que o *software* deve fazer. Para cada produto existem também diversos olhares e perspectivas vindos das partes envolvidas. Para cada caso existe uma forma mais adequada de descobrir os requisitos de um produto de *software*.

Neste capítulo você vai estudar a identificação das fontes de informação dos requisitos de um projeto de *software* em função do tipo de projeto e seu contexto. Vai também conhecer as diversas técnicas para a elicitação de requisitos e quando elas melhor se aplicam, de modo a escolher a melhor técnica para cada contexto de desenvolvimento de *software*.

1 Quais são as fontes de informação dos requisitos?

Existem diferentes tipos de produtos de *software*; cada um implica diferentes tipos de requisitos e, portanto, diferentes fontes de informação para a obtenção desses requisitos. Sistemas de informação que apoiam as áreas administrativas de uma empresa são diferentes de jogos de celular, que, por sua vez, são diferentes de um sistema de controle de tráfego aéreo, que são diferentes de um sistema de troca de mensagens. As particularidades de cada tipo de produto e de cada contexto de projeto influenciarão na identificação das fontes de informação.

Quando falhamos em identificar adequadamente as fontes de informação, podemos ter, como consequência, usuários esquecidos e, portanto, não ouvidos, ou requisitos que surgem quando a entrega do produto está sendo realizada. Portanto, quanto melhor for a nossa capacidade de envolver todas as fontes, maior será a chance de sucesso do projeto.

No início de um projeto, diversos *stakeholders* têm um papel importante para determinar variáveis que influenciarão o seu desenvolvimento. Alguns poderão definir **requisitos de projeto**, como orçamento, prazos, premissas,

restrições e escopo. Geralmente, esses são os clientes, investidores, gestores ou o próprio gerente do projeto. Outros, serão responsáveis por determinar **requisitos de processo**, que influenciarão a forma como a equipe de desenvolvimento irá trabalhar. Esses podem ser externos à organização desenvolvedora, como o próprio cliente ou investidor, ou podem ser internos à organização, como a área de processos ou de *compliance*. E haverá um conjunto deles que definirá efetivamente os **requisitos do produto** de *software*, sejam os funcionais ("o que" o sistema deve fazer), sejam os não funcionais ("como" o sistema deve fazer). Esses *stakeholders* são provenientes das mais variadas áreas, como veremos a seguir.

Tipos de fonte de informação

Existem diversas classificações para os tipos de fonte de informação, como veremos a seguir. Pohl e Rupp (2015) definem basicamente os três tipos listados a seguir.

- ***Stakeholders*:** são pessoas ou organizações que influenciam direta ou indiretamente nos requisitos do sistema, como usuários, operadores do sistema, desenvolvedores, arquitetos, clientes e testadores.
- **Documentos:** contêm informações importantes que podem se tornar requisitos, como documentos de ordem legal ou regulatória, normas e padrões, documentos específicos do domínio de negócio ou da própria empresa, como documentos de requisitos de mais alto nível ou documentos de erros em sistemas legados, por exemplo.
- **Sistemas em operação:** sistemas legados, predecessores ou mesmo concorrentes. Os sistemas podem ser a fonte de informações sobre novas funcionalidades desejadas pelos clientes.

Outra forma de compreender as fontes de informação é a proposta por Wiegers e Beatty (2013), que utilizam o conceito de **classes de usuários** para designar um subconjunto de usuários que utilizam as mesmas funções ou serviços do sistema. Essas classes representam também um indicativo da prioridade que será dada aos requisitos. Eles apresentam uma hierarquia para favorecer o entendimento dos conceitos, conforme ilustra a Figura 1.

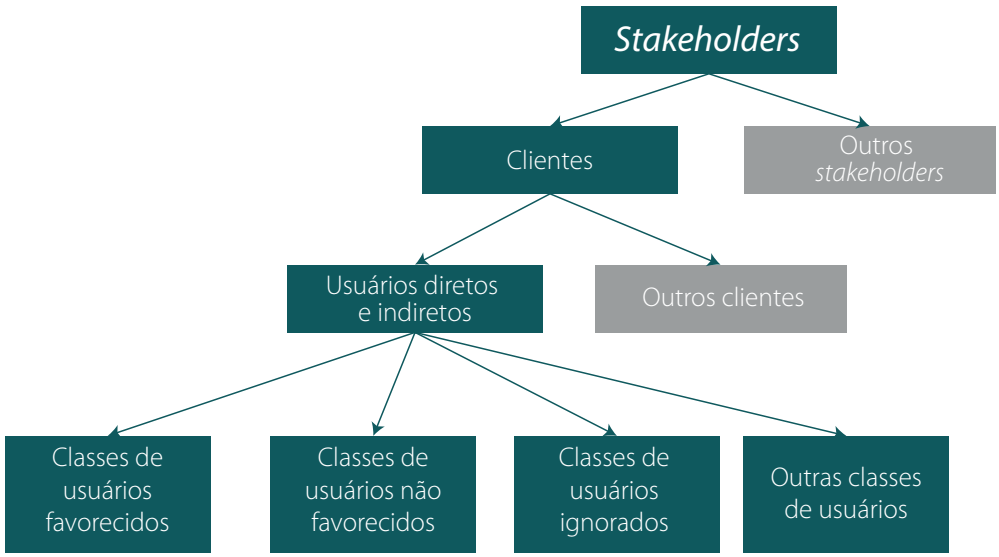


Figura 1. Hierarquia de *stakeholders*, clientes, usuários e classes de usuários.

Fonte: Adaptada de Wiegers e Beatty (2013).

Como se pode observar nessa proposta, os autores utilizam as seguintes definições:

- **Classes de usuários favorecidos:** são aqueles usuários que estão mais diretamente relacionados com a satisfação dos objetivos de negócio do sistema e, portanto, seus requisitos terão maior prioridade do que outros requisitos.
- **Classes de usuários não favorecidos:** são usuários que não devem utilizar o produto, por razões legais, de segurança ou proteção, e que, portanto, devem ser impedidos de fazê-lo por meio de funcionalidades implementadas com essa finalidade (como *hackers*, por exemplo).
- **Classes de usuários ignorados:** são usuários que podem utilizar o produto, porém não são a razão de existir do produto e os seus requisitos terão menor prioridade.
- **Outras classes de usuários:** outros usuários que não sejam os anteriormente mencionados e que terão igual prioridade na definição de requisitos.

É importante lembrar que, no momento da identificação das fontes de informação, todos os potenciais *stakeholders* precisam ser considerados, mesmo aqueles que não utilizarão diretamente as funcionalidades do produto. Por exemplo, há usos indiretos, como aqueles representados por outros *softwares*, que podem receber informações a partir do *software* que está sendo desenvolvido, ou mesmo aqueles *softwares* que realizam funções de forma automática, representando humanos, como os *bots*. A equipe de desenvolvimento também pode ter requisitos não funcionais relacionados, especialmente, à parte interna do sistema, como portabilidade, manutenibilidade, reusabilidade etc. O usuário que utiliza uma funcionalidade do sistema provavelmente não se preocupa com a sua arquitetura interna, mas, para a equipe que será responsável pela manutenção do produto, esses requisitos são relevantes e influenciarão na forma como o produto será concebido e desenvolvido.



Fique atento

Em métodos ágeis como o SCRUM, adota-se o papel do *product owner* (dono do produto), que é responsável por gerenciar o *product backlog* (*backlog* do produto). Suas principais atribuições relacionam-se à responsabilidade pelo gerenciamento das prioridades do *backlog,* bem como garantir que a equipe de desenvolvimento tenha compreendido adequadamente o significado de cada item do *backlog* (SCHWABER; SUTHERLAND, 2013). Nesses casos, é função do *product owner* convergir para uma visão consolidada dos requisitos entre os interesses dos múltiplos *stakeholders* (LEFFINGWELL, 2011).

O grau de envolvimento dos *stakeholders* pode variar de acordo com as diferentes necessidades em relação ao projeto e aos requisitos. Leffingwell (2011) classifica esse envolvimento da seguinte forma:

- ***Stakeholders* que devem ser mantidos informados:** são aqueles que precisam saber do *status* do projeto e precisam ser mantidos informados acerca das decisões, embora nem sempre tomem parte dessas decisões.
- ***Stakeholders* que devem ser consultados:** são aqueles que precisam ser envolvidos devido às suas áreas de *expertise*, como arquitetos, projetistas de interface, *marketing* e especialistas do domínio. Eles devem ser incluídos nas decisões que envolvem a sua área de *expertise*.

- ***Stakeholders* que serão parceiros no desenvolvimento:** são aqueles que precisam ser envolvidos por serem parceiros no processo, como os *product owners* de outros projetos, outras equipes de desenvolvimento, analistas de negócios ou requisitos e provedores de soluções com as quais o sistema deve interagir.
- ***Stakeholders* que controlam os resultados:** são aqueles que tomam as decisões sobre a solução, como executivos, gerentes de liberação, usuários-chave que vão utilizar a solução e *business owners*.

Independentemente de como os *stakeholders* são classificados, é importante que a origem dos requisitos esteja clara e que os potenciais *stakeholders* sejam envolvidos logo no início do processo de elicitação de requisitos. Segundo Wiegers e Beatty (2013), a voz do cliente precisa chegar aos ouvidos dos desenvolvedores. Para que isso seja possível, é necessário que seja identificado corretamente quem precisa ter voz no projeto. Todas as classes de usuários precisam ter alguém que fale por elas.



Fique atento

Pohl e Rupp (2015) sugerem que seja mantida uma lista sempre atualizada com esses envolvidos. Eles argumentam que, caso a lista esteja incompleta ou exista algum *stakeholder* que seja identificado tardiamente, as consequências negativas para o projeto são certas: aspectos importantes podem permanecer não detectados, o objetivo do projeto pode se perder ou custos adicionais podem surgir para a correção dos problemas (retrabalho).

Como descobrir as fontes de informação

Para descobrir quais são as fontes de informação dos requisitos, um bom começo é conversar com o patrocinador do projeto para que ele identifique as áreas de negócios que terão alguma interface com o negócio que está sendo tratado pelo *software,* bem como identifique possíveis leis ou regulamentações que precisam ser atendidas. A partir daí podem surgir inúmeras potenciais fontes de informação que, posteriormente, serão agrupadas dentro de uma mesma classe, uma vez que tenham os mesmos interesses ou que utilizem as mesmas funcionalidades.

Para apoiar a identificação dos *stakeholders* externos e internos, Leffingwell (2011) recomenda que sejam feitas perguntas como as apresentadas no Quadro 1.

Quadro 1. Questões de apoio para identificar fontes de informação

***Stakeholders* externos**	***Stakeholders* internos**
■ Quem utilizará diretamente o sistema? ■ Quem utilizará os resultados daqueles que usam o sistema? ■ Quem será responsável por dar suporte ao sistema? ■ Com quais outros sistemas o nosso sistema irá interagir? ■ Que interfaces devem ser fornecidas? ■ Quem pode fornecer orientações sobre as funcionalidades e os itens de qualidade do sistema (usabilidade, confiabilidade, desempenho e manutenibilidade)?	■ Quem precisa ser consultado sobre o escopo do projeto? ■ Quem contribuiu para o orçamento e o cronograma? ■ Quem gerencia o relacionamento de negócios entre as equipes e o cliente? ■ Quem irá determinar como e quando o sistema deve ser entregue para os clientes? ■ Quem pode afetar ou apoiar politicamente o projeto? ■ Que parceiros dependem do sistema? ■ Quem se preocupa com o processo que será usado no desenvolvimento do sistema?

Fonte: Adaptado de Leffingwell (2011).

Leffingwell (2011) sugere que, para determinados casos, seja utilizado o mapeamento de personas. Uma **persona primária** pode ser entendida como alguém que interage com o *software* e que necessita de uma interface projetada especificamente para ela. Uma **persona secundária** é um usuário que utiliza o *software* com uma interface projetada para outro tipo de usuário. Mapear uma persona significa caracterizar um representante hipotético, genérico, de uma classe de usuários. Personas devem ser caracterizadas considerando características e comportamentos sociais e demográficos, preferências, preocupações e informações similares (WIEGERS; BEATTY, 2013).

2 Técnicas de elicitação de requisitos

O ponto mais crítico da engenharia de requisitos é a **elicitação**. Podemos dizer que ela é o coração do desenvolvimento de requisitos (WIEGERS; BEATTY, 2013). Não se trata apenas de perguntar o que um grupo de usuários deseja, mas, sim, de investigar, instigar, questionar, descobrir, explorar — em resumo, extrair. Não se trata de apenas transcrever o que o usuário relata em uma conversa, mas sim de explorar, de forma colaborativa, todos os aspectos necessários para o entendimento correto do requisito. É um constante exercício de: "e se…", traduzido como: "e se acontecer tal coisa, como o sistema deve reagir?"; "e se não acontecer tal coisa, como o sistema deve tratar?".

Lembre-se, no entanto, de que a tarefa de elicitação de requisitos não é trivial. Os usuários nem sempre sabem explicar as suas tarefas, podem esquecer ou deixar informações importantes de lado ou podem simplesmente não estar disponíveis ou não quererem prestar informações (BOURQUE; FAIRLEY, 2014). É função do analista de requisitos obter o engajamento necessário para que essa atividade ocorra adequadamente, gerando a base para as demais atividades do ciclo de desenvolvimento de *software*. Para que isso seja possível, é necessário selecionar a técnica de elicitação mais apropriada para cada situação específica.

Uma técnica de elicitação de requisitos pode ser compreendida como uma ferramenta para auxiliar o analista de requisitos na condução da etapa de elicitação e compreensão dos requisitos do *software*. Existem diversas técnicas disponíveis; algumas implicam a interação direta com um *stakeholder* humano, enquanto outras se aplicam a outros tipos de fontes de informação.

Independentemente da técnica selecionada, as seguintes etapas de preparação são necessárias:

- **Identificação das fontes de informação:** identificação das pessoas com competência para prestar a informação, bem como das demais fontes de informação, como documentos, elementos multimídia, *softwares* etc., conforme detalhado na seção anterior.
- **Familiarização com o assunto:** buscar conhecer, antecipadamente, as terminologias e os jargões da área de negócio que está sendo tratada, bem como os conceitos-chave do negócio. Pesquisas em documentos do domínio ou uma busca rápida na *web* podem ajudar.

- **Identificação dos objetivos:** identificar claramente os objetivos da elicitação, o nível de informação, os tópicos principais, a relevância, a precedência e as prioridades.
- **Elaboração do cronograma:** elaborar com antecedência o cronograma de atividades necessárias, visando à divulgação dos objetivos e informações logísticas (local, data, horário, materiais necessários).
- **Identificação dos riscos na elicitação:** identificar e documentar os possíveis riscos envolvidos na elicitação de requisitos e as ações que serão adotadas para preveni-los ou mitigá-los.

Neste capítulo, iremos abordar as seguintes técnicas, que envolvem a interação direta ou indireta com fontes de informação humanas:

- entrevista;
- reunião;
- *brainstorming*;
- observação;
- questionário.

Ao final da seção, também abordaremos outras técnicas de elicitação que não envolvem *stakeholders* humanos.

Entrevista

Uma das formas mais usuais de se realizar a elicitação de requisitos a partir de fontes humanas é a entrevista. Trata-se de uma conversa entre duas pessoas, provocada por uma delas, com um objetivo definido. A entrevista proporciona o contato pessoal, que faz com que o entrevistado se sinta parte do processo de construção da solução.

A entrevista facilita a obtenção de informações que, muitas vezes, estão apenas na memória das pessoas, além de informações a respeito dos integrantes da unidade, como suas qualificações, atribuições, utilização de processos, bem como opiniões sobre a unidade e o trabalho. Se uma boa relação é estabelecida com o entrevistado ou com um pequeno grupo de entrevistados, eles podem se sentir mais seguros para falar sobre questões mais delicadas do que em um grupo maior (WIEGERS; BEATTY, 2013).

Como qualquer outra técnica, a entrevista exige uma preparação por parte do entrevistador. É recomendado que ele prepare uma lista de questões mais abrangentes no início e que, progressivamente, ficam mais detalhadas. As questões devem seguir uma ordem lógica e abordar um assunto de cada vez. O roteiro com as questões deve servir apenas para guiar e orientar o processo de entrevista, ele não deve ser usado de forma restritiva, pois a riqueza do processo de entrevista está justamente no oposto. O entrevistador tem que dar espaço para que o entrevistado possa se manifestar livremente sobre seus anseios em relação à solução que está sendo desenvolvida.

A entrevista pode não obter os resultados desejados, caso ocorra uma das seguintes situações:

- falta de preparo do condutor da entrevista;
- falta de preparo do entrevistado;
- tendência do entrevistado em dar respostas agradáveis;
- tendência do entrevistado em focar em palpites ou em respostas falsas;
- resistência do entrevistado com declarações do tipo "não sei", "não conheço".



Saiba mais

Veja aqui algumas dicas adicionais para a condução de entrevistas para a elicitação de requisitos:

- conquiste a confiança do entrevistado se apresentando e explicando claramente a finalidade do encontro;
- tome notas durante a entrevista ou grave-a (se o entrevistado permitir);
- se possível, utilize o papel do redator (liberando você para focar na conversa com o entrevistado);
- deixe para analisar criticamente depois (não corrija falhas, faça críticas ou discuta);
- inicie com questões mais abertas, tipo "como é o trabalho que você realiza?";
- evite termos muito técnicos e tente se aproximar do jargão da área;
- evite perguntas com respostas apenas do tipo "sim" ou "não", que podem limitar a participação do entrevistado;
- se a entrevista se desviar, reconduza, com habilidade, para o fim programado;
- se o entrevistado se recusar a fornecer dados, obtenha a cooperação de outros colegas em papel similar ou, em último caso, do superior hierárquico do entrevistado;
- esclareça as possíveis preocupações por parte do entrevistado;
- dê abertura para novos contatos, da sua parte e da parte do entrevistado;
- valide as informações obtidas com relação ao objetivo previamente estipulado.

Embora a entrevista geralmente seja conduzida de forma presencial, hoje já é muito comum que ela também seja realizada *on-line*, usando ferramentas de comunicação. O único cuidado é garantir que a tecnologia de suporte (ferramenta, rede, internet) funcione adequadamente, evitando as interrupções, que prejudicam o andamento da conversa. Interrupções por falha na tecnologia fazem com que se perca o ritmo da conversa e se gaste tempo para retomar ao ponto anterior.

Wiegers e Beatty (2013) ressaltam a importância de praticar a escuta ativa em todas as interações com os usuários. Isso significa manter uma postura corporal receptiva, ter paciência, dar *feedback* verbal e questionar pontos que não ficaram claros. Além disso, outra forma de demonstrar a escuta ativa é parafrasear o que o entrevistado disse, demonstrando que entendeu a mensagem.

Reunião

Outra forma bastante comum de elicitação de requisitos é a reunião. Uma reunião é o intercâmbio de ideias, sugestões e opiniões entre determinados indivíduos, visando à aceitação de um ponto de vista (esclarecimento, conclusão, decisão, linha de ação etc.) por parte dos participantes.

A reunião é uma comunicação direta entre os participantes e permite a integração do grupo em torno do mesmo assunto, formando uma equipe de trabalho. Ela é usada para a coleta de sugestões e críticas dos participantes (discussões, acertos, acordos e busca de consenso e alternativas); aglutinação e intercâmbio de experiências; e definição de problemas que devem ser tratados pela solução, explorando as suas causas e efeitos. Ela permite o envolvimento das pessoas para a tomada de decisão em grupo.

Assim como a entrevista, a reunião exige uma preparação por parte de quem vai conduzi-la. É preciso organizar a agenda dos tópicos que serão tratados, bem como o tempo estimado para discutir cada item. O ambiente no qual a reunião ocorrerá deve ser amplo, ventilado e iluminado. O espaço deve ser suficiente para acomodar todos os participantes. Material de apoio deve ser providenciado, como quadro branco e canetas para anotações, computador, projetor, tela de projeção e demais materiais de escritório.

A reunião tem estes três momentos:

- **Abertura:** apresentação dos participantes, dos objetivos da reunião, das regras de condução e da duração prevista, bem como definição do redator da ata de reunião. Caso seja uma reunião de continuidade, a ata da reunião anterior deve ser lida e os pontos pendentes, tratados.
- **Abordagem dos tópicos:** exposição dos problemas e/ou questões que se deseja resolver no decorrer da reunião; abertura da discussão e ponderação até a seleção da alternativa mais adequada; busca pela decisão consensual, evitando os conflitos.
- **Conclusão:** definição de quem é responsável pelas ações decorrentes da reunião e como será retornado o *feedback* para os participantes.

Para que uma reunião com o objetivo de elicitação de requisitos seja bem-sucedida, é importante que o analista de requisitos que vai conduzi-la seja cuidadoso ao se preparar. Existem algumas recomendações que podem ajudar:

- encaminhe a agenda para os participantes com antecedência, para que possam se planejar (assunto, participantes, horário, local);
- o término deve ocorrer exatamente na hora marcada, pois as pessoas têm outros compromissos profissionais e, principalmente, pessoais;
- se necessário, faça a reunião em outro local, afastado do ambiente de trabalho, para evitar interrupções;
- não defenda ao extremo os seus dados, prefira levar o grupo a deduzir que eles estão corretos a partir das informações objetivas;
- não mostre contrariedade quando o grupo rejeitar suas ideias;
- evite impasses, pois, em uma discussão, não devem existir vencedores nem derrotados — isso é péssimo para a continuidade do projeto;
- não mude de opinião apenas para impedir o conflito, siga os seus objetivos com coerência;
- encaminhe as decisões por meio de consenso, nunca por votação ou barganha;
- ao concluir a reunião, agende data, horário e local da próxima, se necessário;
- se possível, procure registrar os acontecimentos diretamente em uma ata ou memória da reunião e obtenha as assinaturas ao final da reunião, evitando assim questionamentos posteriores quanto ao seu conteúdo.

Embora tudo tenha sido preparado com antecedência, alguns imprevistos podem acontecer e a reunião pode não ser bem-sucedida caso ocorra uma das seguintes situações:

- se houver uma possível dominação da reunião por um só indivíduo, cujo talento, personalidade ou conhecimento inibem os demais participantes;
- se houver conformidade de alguns participantes, aceitando a opinião da maioria para se sentirem parte do grupo ou para evitar o conflito (isso pode inibir soluções novas, arrojadas e diferentes);
- se houver faltas, atrasos e saídas antecipadas;
- se o ambiente físico for inadequado, levando ao desconforto ou distrações (ambiente com muito barulho, por exemplo);
- se houver participação de pessoas não relacionadas ao assunto, que não contribuem e ainda podem atrapalhar os resultados.



Saiba mais

Veja aqui algumas dicas de perguntas para a condução de reuniões para a elicitação de requisitos:

- **Pergunta dirigida:** pergunta dirigida especificamente a um participante. Ideal para lidar com pessoas tímidas. Por exemplo: "Gostaríamos de ouvir a opinião de Fulano sobre o assunto"; "Por favor, Sr. Fulano, poderia nos dizer alguma coisa?".
- **Pergunta geral:** pergunta dirigida a todos os participantes, com a finalidade de estimular a discussão. É um meio de reanimar discussões que, repentinamente, esfriaram. Por exemplo: "Que mais poderíamos considerar a respeito do que foi exposto?"; "Que outras medidas poderiam ser tomadas nesse caso?"; "Que alternativas vocês sugerem?".
- **Pergunta redistribuída:** pergunta que é retornada por quem a recebeu para um dos demais participantes ou a todo o grupo. Por exemplo: "É uma boa pergunta, como os senhores responderiam?"; "Ninguém melhor do que Fulano, que entende desse assunto, para responder a essa pergunta".
- **Reversa:** pergunta endereçada ao condutor, que a devolve a quem a formulou. Esse processo exige certo tato, porque pode gerar antagonismo ou embaraço no participante, que pode, receando que isso se repita, optar por não mais se manifestar.

Brainstorming

O *brainstorming* consiste em uma "tempestade de ideias", sem julgamentos ou análises, que ocorre em um ambiente descontraído e informal. Sentar-se sozinho ou com um amigo para pensar em uma nova ideia não é exatamente um *brainstorming*. A aplicação da técnica requer preparação e envolvimento de mais pessoas.

Inicialmente, deve ser selecionada uma dinâmica para o entrosamento dos participantes, que permita que o ambiente fique descontraído e propício à criatividade. É comum que seja alguma brincadeira simples, com cunho divertido, realizada em pé ou se movimentando. Esse aquecimento não deve tomar muito tempo do evento.

O material necessário para o *brainstorming* também deve ser providenciado com antecedência e pode ser constituído de notinhas adesivas coloridas (tipo Post-it) e canetas coloridas. A quantidade vai depender da quantidade de pessoas e dos objetivos do *brainstorming*. Lembre-se de que a base é a geração de muitas ideias, e cada uma deve ser escrita em uma notinha.

A seção de *brainstorming* é composta por três fases: aquecimento (com a dinâmica selecionada), ideação (com a geração das ideias) e encerramento (com o agrupamento das ideias). Opcionalmente, pode ser realizada, na última etapa, uma breve análise ou votação das melhores ideias.



Fique atento

Algumas questões devem ser observadas para que o *brainstorming* seja realizado com sucesso:

- o *brainstorming* não se aplica a grupos muito fechados e formais;
- o clima precisa ser descontraído e não pode haver críticas, julgamentos e gozações, pois isso inibe o fluxo de ideias;
- quando o grupo for muito grande, o ideal é dividi-lo em subgrupos com não mais do que 8 pessoas cada (o número depende do contexto e dos perfis).

Observação

Como o próprio nome sugere, esta é uma técnica na qual o analista de requisitos vai a campo para poder observar como o trabalho ocorre. Esta técnica utiliza os sentidos na obtenção de determinados aspectos da realidade. Não consiste apenas em ver e ouvir, mas em examinar fatos ou fenômenos que se deseja estudar.

A observação pode ser usada para confirmar informações obtidas das entrevistas, questionários ou documentos, comparando-as com a realidade. Pode-se levantar volumes, acessar outras fontes de informações (manuais de procedimentos etc.) e levantar o fluxo de documentos ao longo das tarefas. Ela é capaz ainda de ajudar a ter ideias sobre pontos nos quais o novo *software* pode reduzir gargalos e minimizar trabalhos realizados de forma manual.

A principal limitação da observação é a interferência, mesmo que não intencional, no ambiente de trabalho. A presença de um observador sempre provoca perturbações nas pessoas, o que pode alterar as condições reais em que o trabalho é realizado, por mais discreto que seja o observador. Outra questão importante é que o observador tem seu próprio ponto de vista, ou seja, a sua ótica pessoal, e pode tender a criar impressões favoráveis ou desfavoráveis. Pode haver eventos que ocorrem ocasionalmente e sua ocorrência espontânea não pode ser prevista, o que impede, muitas vezes, o observador de presenciar o fato. E, uma última limitação, é que a observação consome tempo e certamente não será viável para ser utilizada em cada tarefa que o usuário realiza.

O tempo de duração de uma observação pode variar, de acordo com o que se pretende observar. É importante certificar-se que o fenômeno que se deseja observar irá ocorrer durante o período da observação. Se a necessidade for de observar algo que ocorre apenas em momentos de pico, por exemplo, é necessário que sejam identificados os períodos em que esses picos tendem a acontecer, para que o observador possa estar presente.

O observador deve tomar notas de forma discreta, buscando não despertar nas pessoas a sensação de estarem sendo vigiadas. Gravações de áudio e vídeo só podem ser realizadas com autorização.

Há casos em que é possível realizar uma observação menos passiva e mais ativa, na qual uma interação com o executor da tarefa é possível. Isso pode ser útil para compreender por que ele tomou aquela decisão naquele momento e com base em que informações a decisão foi tomada.



Fique atento

A observação requer um olhar atento e muita discrição. Eis o que você deve observar no ambiente:

- Como ocorre o fluxo de atividades entre os participantes do processo?
- Existem momentos de pico de atividades?
- Existem problemas causados pela tecnologia ou pela ausência dela?
- O que acontece quando a tecnologia falha (cai a rede, por exemplo)?
- Quais são os fluxos alternativos e casos que requerem um tratamento especial?

Questionário

A técnica de questionário, também conhecida como *survey* ou pesquisa de opinião, nada mais é do que a aplicação de um instrumento na forma de questionário, que tem por objetivo coletar informações de fontes, que podem ser em grande volume e estar distantes fisicamente. O questionário pode ser a porta de entrada para outras técnicas, por exemplo, levantando as principais dores dos usuários com o processo ou o sistema atual (WIEGERS; BEATTY, 2013).

O questionário é composto por um conjunto de perguntas com respostas objetivas ou abertas, dispostas em sequência lógica e progressiva. O objetivo do questionário é obter informações que servirão de base para tratamentos estatísticos. Lembre-se de que, se forem poucas pessoas, talvez seja melhor realizar uma entrevista. Tenha em mente também que os questionários são frios e impessoais e não permitem a mesma riqueza de outras técnicas.

Embora, à primeira vista, possa parecer que utilizar um questionário seja uma atividade simples, na realidade, não é. Elaborar questões é uma tarefa que exige conhecimento e experiência. Primeiro, porque as pessoas não gostam de responder questionários e, portanto, ele deve ser curto e fácil de responder. Segundo, porque exige habilidade para não introduzir viés e nem direcionar as respostas. E, por fim, as perguntas devem permitir que sejam obtidas as informações que o analista de requisitos necessita efetivamente.

A preparação do questionário envolve elaborar as questões, estimar o tempo de resposta, simular previamente as análises e realizar um teste piloto. Esse teste piloto deve permitir identificar se as pessoas conseguem compreender adequadamente o que está sendo perguntado e se o tempo de preenchimento está compatível com o estimado. Ajustes podem ser feitos após o teste piloto.

Depois da etapa de preparação é que o questionário pode ser distribuído para o público-alvo, estipulando-se a data esperada para retorno. Lembretes para resposta podem ser enviados no período.



Fique atento

Para aumentar as chances de sucesso do seu questionário, fique atento a estas dicas:

- adicione uma breve introdução, que esclareça os pontos principais;
- utilize uma linguagem clara e simples para elaborar as perguntas;
- utilize frases concisas, sem prejudicar o entendimento da questão;
- utilize termos que evitem a dupla interpretação;
- dê preferência para as questões objetivas e que possam ser facilmente tabuladas, fornecendo conclusões objetivas sobre o trabalho em andamento;
- revise se as alternativas cobrem todas as possibilidades que deseja investigar;
- não deixe um espaço muito grande para as respostas das questões abertas, de modo a evitar longos discursos do respondente;
- utilize perguntas encadeadas (para testar a coerência das respostas) e perguntas complementares (fará esgotar o assunto).

Outras técnicas

As técnicas descritas até aqui se aplicam à elicitação de requisitos realizada quando existem classes de *stakeholders* que podem ser representadas por uma pessoa, com a qual se pode fazer contato direto (como entrevistas e reuniões) ou indireto (como no caso do questionário). Existem ainda outras técnicas derivadas do *design thinking* que podem ser utilizadas quando as fontes são humanas, mas elas não serão objeto deste capítulo.

Existem alguns tipos de requisitos que são provenientes de outras fontes não humanas, como, por exemplo, **leis ou normativas de órgãos regulamentadores**. No segmento bancário, por exemplo, temos as normativas provenientes do Banco Central, do Conselho Monetário Nacional, da Comissão de Valores Mobiliários (CVM) etc. Temos ainda órgãos que regulamentam outros setores como a Agência Nacional de Saúde (ANS), a Agência Nacional de Telecomunicações (ANATEL), a Agência Nacional de Energia Elétrica (ANEEL) e diversos outros. Essas leis ou normativas precisam ser lidas e interpretadas pelo

analista de requisitos junto com os especialistas do negócio. As regras impostas tornam-se requisitos que devem ser atendidos pelo produto de *software*.

Algumas vezes, o produto está sendo desenvolvido para substituir um *software* anterior, que pode não ter documentação associada e, portanto, a única fonte dos requisitos é o próprio *software* que está sendo substituído. Nesses casos, os requisitos podem ter que ser extraídos por meio da **execução simulada de tarefas** (funcionalidades) ou da **leitura do código-fonte**. Além disto, pode ainda haver um *backlog* de sugestões de melhoria pendentes do sistema anterior, que pode servir de base para as funcionalidades do novo produto. Abordagem similar é utilizada quando se quer obter requisitos a partir do **produto dos concorrentes**, com a diferença de que, normalmente, não se tem acesso ao código-fonte.

3 Seleção da técnica de elicitação de requisitos

Na fase de elicitação de requisitos nunca se utiliza apenas uma técnica e nem somente aquela que se conhece, mas todas as que forem necessárias ou apropriadas para cada caso. Algumas dicas para aplicar a técnica mais adequada e obter os melhores resultados de seu uso são descritas no Quadro 2.

Quadro 2. Seleção das técnicas de elicitação de requisitos

Técnica	Quando usar
Entrevista	■ Quando existem pessoas que têm o conhecimento necessário sobre o negócio e que estão disponíveis para serem entrevistadas. ■ Para captar as informações subjetivas (desabafos, sentimentos, pontos de vista) que podem ser úteis para a definição de requisitos do sistema. ■ Para identificar o fluxo de trabalho e de documentos.

(Continua)

(Continuação)

Quadro 2. Seleção das técnicas de elicitação de requisitos

Técnica	Quando usar
Reunião	■ Para obter uma resposta rápida de várias pessoas sobre determinado assunto. ■ Quando existem problemas ou questões a serem esclarecidos, compartilhados e expostos com os demais participantes. ■ Para envolver o grupo da tomada de decisão. ■ Para resolver situações de conflito (busca de consenso). ■ Para fazer levantamento de alternativas e soluções para determinado problema. ■ Para analisar, avaliar e resolver problemas que envolvam pessoas de áreas ou empresas distintas. ■ Por exigência legal da empresa ou do governo.
Brainstorming	■ Quando o ambiente da empresa é informal e propício para o desenvolvimento de atividades criativas. ■ Quando se deseja encontrar soluções inovadoras e criativas. ■ Quando se deseja lançar novos produtos no mercado.
Observação	■ Quando o fluxo de papéis e de trabalho é relevante. ■ Quando problemas de desempenho estão relacionados à forma de executar o trabalho. ■ Quando a influência do ambiente real é importante.
Questionário	■ Se não houver tempo suficiente para entrevistar todas as pessoas relevantes. ■ Se as informações puderem ser adequadamente trabalhadas para fins estatísticos. ■ Se as pessoas estão situadas em pontos geográficos muito distantes. ■ Se, para prestar informações, houver necessidade de consultar arquivos físicos.

Wiegers e Beatty (2013) compilaram diversas informações e recomendações para a seleção da melhor técnica de elicitação de requisitos de acordo com as características dos projetos. Elas estão representadas no Quadro 3.

Quadro 3. Seleção das técnicas de elicitação de requisitos de acordo com Wiegers e Beatty (2013)

	Entrevista	***Workshop***	**Grupo Focal**	**Observação**	**Questionário**	**Análise de interface de sistemas**	**Análise de interface de usuário**	**Análise de documentos**
Software para o mercado	×		×		×			
Software interno para a organização	×	×	×	×		×		×
Software para substituir outro	×	×		×		×	×	×
Melhorias em sistemas existentes	×	×				×	×	×
Novo *software*	×	×				×		
Implementação de pacote de *software*	×	×		×		×		×
Sistemas embarcados	×	×				×		×
Stakeholders distribuídos geograficamente	×	×			×			

Fonte: Adaptado de Wiegers e Beatty (2013).

Como se pode observar no Quadro 3, os autores consideram duas formas de elicitação que não foram abordadas neste capítulo: grupo focal e *workshop*. Mais informações sobre essas formas de elicitar requisitos podem ser obtidas em Wiegers e Beatty (2013). Não considere esta lista como definitiva e exaustiva. As técnicas podem ser combinadas de acordo com suas necessidades em cada projeto.



Referências

BOURQUE, P.; FAIRLEY, R. *SWEBOK:* guide to the *software* engineering body of knowledge version 3. [*S. l.*]: IEEE Computer Society, 2014.

LEFFINGWELL, D. *Agile software requirements:* lean requirements practices for teams, programs, and the enterprise. Upper Saddle River: Pearson Education, 2011.

POHL, K.; RUPP, C. *Requirements engineering fundamentals:* a study guide for the certified professional for requirements engineering exam, foundation level, IREB Compliant. 2. ed. Santa Barbara: Rock Nook, 2015.

SCHWABER, K.; SUTHERLAND, J. Guia do SCRUM: um guia definitivo para o SCRUM: as regras do jogo. [*S. l.*], 2013. Disponível em: https://www.scrumguides.org/docs/scrumguide/v1/Scrum-Guide-Portuguese-BR.pdf. Acesso em: 31 jan. 2020.

WIEGERS, K. E.; BEATTY, J. *Software requirements*. 3. ed. Redmond: Microsoft Press, 2013.

Leitura recomendada

PRESSMAN, R.; MAXIM, B. *Engenharia de software*: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

Parte 4

Aplicação de Técnicas de Elicitação de Requisitos de Software

O conteúdo deste livro é disponibilizado por SAGAH.

SAGAH SOLUÇÕES EDUCACIONAIS INTEGRADAS

Aplicação de técnicas de elicitação de requisitos de *software*

Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Planejar a aplicação das técnicas de elicitação de requisitos de projetos de *software*.
- Aplicar técnicas de elicitação de requisitos de projetos de *software*.
- Organizar os requisitos de um projeto de *software*.

Introdução

Já nos habituamos com os benefícios e os efeitos positivos do uso das tecnologias em nosso dia a dia. Nossos *smartphones* permitem que realizemos a maior parte das atividades cotidianas sem sair de casa. Antigamente, para pagar uma conta, era necessário se deslocar até o banco, aguardar pacientemente em uma fila e então efetuar o pagamento com a ajuda do caixa, que era o funcionário do banco responsável por realizar essa atividade, usando os equipamentos que também eram do banco. Embora esta opção ainda exista, hoje a maioria de nós faz tudo isso sem sair de casa, bastando apontar o celular para um código de barras, que, automaticamente, já reconhece todos os dados para que o pagamento seja realizado. Nem a senha precisamos mais digitar: basta olhar para o celular; ele reconhece nossa face e autoriza a operação. Tudo isso acontece em menos de 5 minutos e no conforto do lar ou do escritório.

Esse é apenas um dos muitos exemplos de facilidades que a tecnologia nos traz. Mas você já parou para pensar como são concebidos esses *softwares*? Como são identificadas as funcionalidades que um aplicativo deve ter? Quem define e como são descobertos os requisitos que dão origem a um produto de *software*? Quem prioriza o que deve ser entregue

na primeira versão do produto? E nas demais? Quem diz como deve acontecer a integração com outros *softwares*, ou até mesmo *hardwares*, de modo a facilitar ainda mais a nossa vida?

A resposta a essas questões se encontra na engenharia de requisitos, ou, mais precisamente, na etapa de elicitação de requisitos. É nessa etapa que são identificadas as fontes de informação, ou seja, de onde serão obtidos os requisitos. Haverá usuários que possam definir o que querem? Há leis ou regulamentações que precisam ser obedecidas? Há investidores que pagam pelo projeto e têm voz ativa na sua definição? É nessa etapa também que são selecionadas, em função do contexto, as técnicas de elicitação de requisitos que serão utilizadas.

Neste capítulo, você vai estudar o planejamento da aplicação das técnicas mais usadas de elicitação de requisitos em projetos de *software*, vendo algumas dicas para aumentar as suas chances de sucesso. Vai também ler sobre como aplicar essas técnicas e como organizar as informações obtidas para produzir os requisitos do *software*.

1 Planejamento da aplicação das técnicas de elicitação de requisitos

A principal missão da etapa de elicitação de requisitos, como o próprio nome diz, é descobrir um conjunto de requisitos que seja compreendido por todos os envolvidos e que esteja apto para seguir para as próximas etapas. Independentemente de a equipe utilizar uma abordagem ágil de desenvolvimento, como o Scrum, ou uma abordagem mais tradicional, os requisitos precisam ser definidos para que os desenvolvedores possam transformá-los em programas, ou seja, em linhas de código implementadas.

Dependendo do tipo de *software* que está sendo desenvolvido, diferentes fontes de informação serão necessárias. Se estivermos falando de um sistema de informação, as fontes podem ser os usuários das diversas funcionalidades que o *software* irá prover. Se estivermos falando de um sistema composto por *hardware* e *software*, então precisamos compreender se os serviços serão fornecidos diretamente para um usuário humano ou se serão parte de algum sistema de controle de equipamentos. Se for um jogo eletrônico, pode ser que as fontes de informação sejam outros jogos similares que queremos superar no mercado.

Uma vez identificadas essas fontes de informação, é hora de selecionar as técnicas de elicitação a serem aplicadas. Existem diversas técnicas que podem ser utilizadas tanto isoladamente quanto combinadas entre si. Algumas delas são reunião, entrevista, observação, questionário, JAD (*joint application design*), *brainstorming*, análise de sistemas anteriores, análise de documentos (leis, regulamentações, manuais, editais de licitação), grupo focal, entre outras. Uma comparação entre os atributos de algumas dessas técnicas pode ser encontrada no Quadro 1.

Quadro 1. Comparação das técnicas de elicitação de requisitos

	Entrevista	**JAD**	**Questionário**	**Análise de documentos**	**Observação**
Tipo de informação	■ Como é ■ Melhorias ■ Como será	■ Como é ■ Melhorias ■ Como será	■ Como é ■ Melhorias	■ Como é	■ Como é
Profundidade da informação	Alta	Alta	Média	Baixa	Baixa
Abrangência da informação	Baixa	Média	Alta	Alta	Baixa
Integração da informação	Baixa	Alta	Baixa	Baixa	Baixa
Envolvimento do usuário	Médio	Alto	Baixo	Baixo	Baixo
Custo	Médio	Baixo-médio	Baixo	Baixo	Baixo-médio

Fonte: Adaptado de Dennis, Wixon e Roth (2012).

As próximas seções apresentarão a forma como podemos nos planejar para aplicar três dessas técnicas: questionário, entrevista e observação.

Questionário

O questionário é um instrumento que permite a coleta de informações quando se tem um grande grupo de usuários ou quando eles estão geograficamente distantes, dificultando a aplicação de técnicas que demandam interações pessoais, como entrevistas ou reuniões, por exemplo.

A Figura 1 apresenta os passos que devem ser seguidos para a aplicação da técnica de questionário. Inicialmente, é fundamental **determinar os objetivos da pesquisa**, ou seja, da aplicação do questionário, de forma clara. O questionário pode ser utilizado para levantar os problemas com sistemas existentes, as chamadas "dores do usuário", que podem servir de base para o planejamento dos requisitos do sistema que está sendo desenvolvido. Pode ainda apoiar a priorização dos requisitos identificados, levando em consideração as possíveis diferenças entre usuários geograficamente dispersos, que podem vivenciar cenários distintos e ter prioridades diferentes. Se um novo produto será lançado no mercado, o objetivo do questionário pode ser "Identificar a potencial aceitação do produto no mercado".

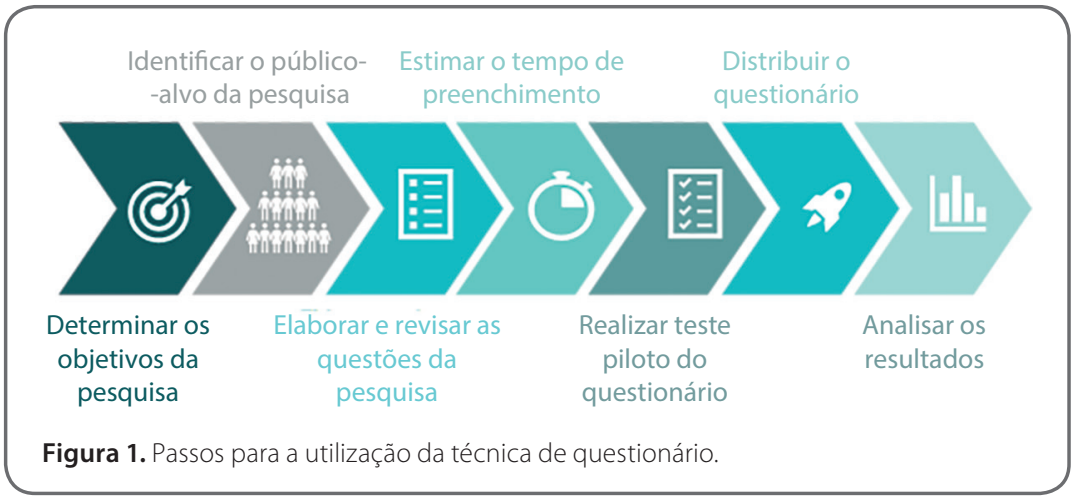


Figura 1. Passos para a utilização da técnica de questionário.

Em seguida, é necessário **identificar o público-alvo da pesquisa**. Aplicar o questionário para o público-alvo incorreto pode levar ao não atingimento dos objetivos da pesquisa. O público-alvo pode ser constituído de membros internos ou externos à organização. As questões serão elaboradas em função do perfil desse público e será definida a forma mais adequada de aplicação.

Um questionário pode tanto ser aplicado de forma *on-line*, na qual o respondente preenche sozinho, quanto pode ser realizado na forma de uma entrevista estruturada, na qual um agente realiza as perguntas, seguindo o roteiro.

A etapa mais complexa da técnica de questionário é **elaborar e revisar as questões** que serão utilizadas e como as respostas serão tabuladas e interpretadas. Se elas não forem bem escritas pode haver interpretação incorreta por parte dos respondentes e os resultados poderão conter algum viés. Se o questionário for *on-line*, não haverá uma pessoa para esclarecer as dúvidas do respondente. Se o questionário for muito complexo ou se for muito extenso, as pessoas também não irão respondê-lo.

Veja a seguir os tipos de questão que podem aparecer em um questionário (BARRIBEAU *et al.*, 2020):

- **Escalas de valores:** peça aos respondentes que valorem alguma coisa, como uma ideia, um conceito, um indivíduo, um programa, um produto etc., com base em um formato de escala fechado, usualmente uma escala de 5 pontos. Por exemplo, uma escala Likert apresenta aos respondentes uma série de afirmações em vez de questões, e os respondentes são solicitados a responder em que grau concordam ou discordam da afirmação.
- **Escalas de ranqueamento:** peça aos respondentes para ranquear um conjunto de ideias, coisas etc. Por exemplo, o pesquisador pode fornecer aos respondentes uma lista de requisitos e pedir que estabeleçam a prioridade entre eles, na qual a prioridade 1 é dada ao requisito mais prioritário. Isso é mais difícil de usar do que uma escala de valores. As escalas de ranqueamento vão requerer mais tempo e não podem ser facilmente usadas quando se tem um entrevistador humano. Uma vez que elas são mais difíceis, requerem maior esforço dos respondentes também.
- **Escalas de estimativa de magnitude:** peça aos respondentes para proverem um valor numérico estimado. Por exemplo, pode ser perguntado aos respondentes: "Uma vez que o requisito X é o menos prioritário, vamos lhe atribuir o valor 10. Se você achar que um requisito é 20 vezes mais prioritário do que o requisito X, você irá atribuir o valor 200, e assim por diante. Então, comparado com o requisito X, que tem o valor 10, quão prioritário é o requisito Y?" Essas escalas obviamente são muito difíceis para os respondentes. No entanto, elas podem ajudar a explicar a variância em escalas ordinais.

- **Questões divididas ou desdobradas:** comece com uma questão geral e então vá completando com questões de aprofundamento.
- **Questões de afunilamento:** guie os respondentes nas questões e conceitos complexos, usando uma série de questões que vão, progressivamente, estreitando-se para uma questão específica. Por exemplo, os pesquisadores podem começar com uma questão geral aberta e então progredir para questões mais específicas, fechadas, com preenchimento obrigatório.
- **Questões de afunilamento invertido:** pergunte aos respondentes uma série de questões que vão de aspectos específicos para aspectos mais gerais. Por exemplo, os pesquisadores podem perguntar aos respondentes primeiro as questões mais específicas e fechadas e depois as questões mais gerais e abertas. Essa técnica funciona bem quando os respondentes não têm conhecimento sobre determinada área ou quando eles não têm uma opinião formada sobre determinado assunto.
- **Questões fatoriais:** utilize histórias ou vinhetas para estudar os processos de julgamento e a tomada de decisão. Por exemplo, faça questões do tipo "Você está em uma situação perigosa em um prédio em chamas. Você sai imediatamente do edifício ou você vai aos andares superiores para acordar os demais moradores?". Isso pode ser usado também em situações reais de tomada de decisão no trabalho, para capturar as decisões de negócio.

As dicas a seguir foram compiladas e estendidas a partir do trabalho de Wiegers e Beatty (2013):

- Forneça opções de resposta que cubram todo o conjunto de respostas possíveis e que sejam de interesse da pesquisa. Por exemplo, se você quiser saber o grau de instrução do respondente, mas o questionário só será distribuído para pessoas com curso superior, por exemplo, então as alternativas "ensino fundamental" e "ensino médio" não serão de interesse e não precisam aparecer. Neste caso, apenas as opções a partir do "ensino superior" interessam.

- Faça com que as alternativas de resposta sejam mutuamente exclusivas (que não haja sobreposição dos intervalos de valores, por exemplo) e exaustivas (liste todas as possíveis escolhas e/ou deixe um espaço para os respondentes inserirem outra resposta que você não tenha previsto). Especial atenção deve ser prestada aos limites dos intervalos: se um intervalo é de 0 a 7 e o outro é de 7 a 10 existe uma sobreposição em relação ao 7. Então é importante definir se será de 0 a 7 e de 7,1 a 10 ou se será de 0 a 6,9 e de 7,0 a 10, ou seja, é preciso definir para que lado vai o 7,0.
- Não elabore uma questão de forma que implique em uma "resposta correta". Lembre-se de que você não está testando o usuário, está coletando as suas opiniões e/ou vivências.
- Se forem usadas escalas, certifique-se de utilizá-las de forma consistente no questionário todo, inclusive na ordem de apresentação. Se em uma questão o maior valor aparece mais à esquerda e os demais em ordem decrescente, lembre-se de manter a mesma lógica para outras questões que usam escala similar.
- Utilize questões fechadas com duas ou mais opções se você deseja que os resultados sejam analisados estatisticamente. Questões abertas permitem que os usuários respondam da forma que desejarem, e será difícil encontrar os pontos comuns entre os resultados.
- Pense em consultar um especialista em projeto e aplicação de questionários para certificar-se de que você fez as perguntas corretas para as pessoas corretas.
- Sempre teste um questionário antes de distribuí-lo. É frustrante descobrir muito tarde que uma questão foi escrita de forma ambígua ou que uma questão importante foi esquecida. Lembre-se ainda de testar todos os caminhos lógicos que o questionário possa ter (quando houver possibilidades de desvio em função de alguma resposta).
- Não faça perguntas demais ou as pessoas não vão responder, ou, pior, podem responder as últimas questões sem prestar atenção.

A seguir estão compilados alguns problemas que podem surgir na elaboração de um questionário e que devem ser evitados, de acordo com o guia de escrita da Universidade do Colorado, escrito por Barribeau *et al.* (2020):

- **Achar que todas as pessoas têm uma opinião:** nem sempre as pessoas sabem responder uma pergunta ou têm uma opinião a respeito de um tema. Por isso, é útil prever uma alternativa "não sei responder" ou "não tenho uma opinião".
- **Viés da ordem das respostas:** tendência do respondente de não analisar todas as opções e escolher a mais fácil (muitas vezes a primeira, ou a última). Isso pode ocorrer se for apresentada uma lista muito longa de alternativas ou muitas opções em uma escala de valores.
- **Conjunto de respostas:** quando uma lista muito longa de itens com respostas simples como "sim/não" ou "concordo/discordo" for apresentada, pode ser que o respondente opte por preencher todas com a mesma alternativa, em vez de ler uma a uma. Nesse caso, a lista longa deve ser evitada.
- **Efeito telescópio:** pode ocorrer quando o respondente afirma que um evento ocorreu mais recentemente do que na realidade. Para ajudar, o questionário poderia ter pontos de referência (eventos) que ajudem o respondente a se localizar no tempo.
- **Efeito fadiga:** acontece quando os respondentes se cansam de responder. Uma dica aqui é variar os tipos de questão ou colocar questões mais fáceis no final.

Uma vez elaboradas as questões, deve-se **estimar o tempo de preenchimento**. É importante lembrar que essa estimativa deve levar em consideração o perfil planejado dos respondentes.



Fique atento

Não estime o tempo de preenchimento do questionário considerando que é você quem está respondendo. Sempre leve em consideração o perfil planejado dos respondentes.

A partir daí, chegou a hora de **realizar o teste piloto do questionário** para garantir que ele está pronto para ir a campo. O objetivo principal do teste piloto é avaliar se o questionário está completo, claro e se irá atingir os objetivos pretendidos. As pessoas selecionadas para realizar o piloto devem responder às questões com o objetivo de identificar possíveis problemas. É uma etapa similar à de teste de *software*: os *bugs* do questionário precisam ser identificados e corrigidos antes que o instrumento seja distribuído para o público-alvo. Além de testar as questões, é importante testar também a tabulação dos resultados — isso poderá ajudar a antever problemas nas análises, quando a fase de coleta já estiver encerrada.

Caso sejam detectados problemas no questionário durante o teste piloto, eles deverão ser solucionados antes de **distribuir o questionário** para o público-alvo. Neste momento, deve ser planejada a estratégia para a distribuição. Caso seja optado pelo formato *on-line*, o *link* para o questionário deve ser enviado para o público-alvo com as instruções claras de preenchimento e a data limite. Não se recomenda que seja dado um prazo muito longo, pois é sabido que isso tende a reduzir o engajamento. Quando a data limite estiver se aproximando, um lembrete deverá ser enviado para os respondentes que ainda não preencheram o questionário.

A última etapa é **analisar os resultados**. Neste momento as informações que foram coletadas são interpretadas e analisadas, servindo de subsídio para a elicitação dos requisitos. A forma de organizar os requisitos será tratada nas próximas seções.

Entrevista

A entrevista é a forma mais usual de se realizar a elicitação de requisitos. Nela o analista de requisitos interage com os *stakeholders* relevantes para o projeto, em uma conversa individual ou em pequenos grupos. A grande vantagem da entrevista é a interação que ela permite entre as partes, bem como a obtenção de informações não verbais, que podem ser capturadas por meio de uma conversa pessoal.

A Figura 2 apresenta as etapas para a realização da entrevista. Como você pode observar, assim como nas demais técnicas, tudo começa com a atividade de **identificar *stakeholders*** que serão fontes de informação sobre os requisitos. Esta é uma etapa crucial, pois pode ser desastroso para o projeto descobrir, mais adiante, que um *stakeholder* relevante não foi envolvido.

Um *stakeholder* precisa ser envolvido nas entrevistas quando ele

- for o demandante do projeto;
- tiver informações relevantes sobre o processo de negócio;
- for afetado diretamente pelos resultados do projeto;
- for politicamente importante para o projeto, podendo se transformar em um apoiador ou em um detrator do projeto.

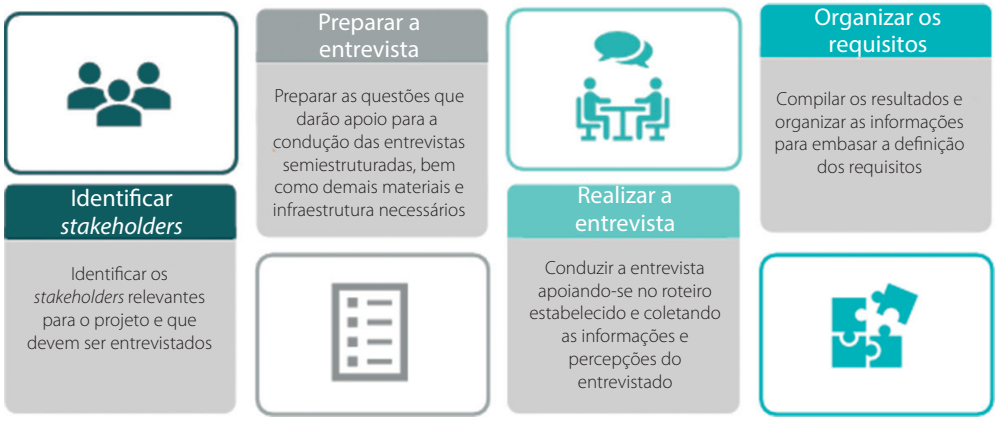


Figura 2. Passos para a utilização da técnica de entrevista.

Uma vez identificadas as fontes de informação e as pessoas que deverão ser entrevistadas, é hora de **preparar a entrevista**. Essa é uma atividade que pode variar, de acordo com o conhecimento e a experiência do analista de requisitos, a complexidade do domínio da aplicação e a disponibilidade de recursos e tempo.

As seguintes atividades são recomendadas como preparação para realizar a entrevista:

- **Compreender o contexto:** caso o analista de requisitos ainda não conheça muito bem o domínio de negócios, é recomendado que busque informações em documentos do projeto, leis e/ou regulamentações do setor, *blogs*, artigos, livros, conversas com colegas mais experientes, sistemas anteriores etc. Um analista de requisitos que não conhece minimamente o domínio do negócio não conseguirá extrair o máximo da entrevista.

- **Elaborar um roteiro semiestruturado:** uma das piores coisas que pode acontecer na entrevista é o analista de requisitos sair com informações incompletas ou muito rasas. Para que isso não aconteça, é importante que um roteiro, ou pelo menos um *checklist*, seja elaborado antecipadamente. Esse roteiro deve conter os principais tópicos a serem tratados, incluindo dúvidas que o analista de requisitos já tenha levantando nas suas pesquisas prévias ou em entrevistas anteriores. O roteiro não será um interrogatório, será apenas um apoio para que o analista de requisitos conduza a entrevista.
- **Preparar material adicional de apoio:** o roteiro vai ajudá-lo na condução das questões, mas pode também ser necessário preparar algum material de apoio, que pode ser diagramas, esquemas, protótipos de interface ou qualquer outra coisa que você julgue oportuno. Figuras e esquemas ajudam as pessoas a se organizar mentalmente sobre o assunto e podem fazer com que a conversa seja mais produtiva.
- **Agendar a entrevista:** o ideal é que uma entrevista dure de uma a duas horas, no máximo. Entrevistas muito curtas podem não ser suficientes para que o tema seja tratado em profundidade. Lembre-se de que os primeiros minutos são dedicados a quebrar o gelo e não a tratar especificamente das questões do projeto. Por outro lado, entrevistas muito longas podem ser cansativas e pouco produtivas para ambos os lados. Evite horários muito próximos do almoço ou do final do expediente, pois as pessoas podem ter compromissos pessoais e ficar ansiosas para encerrar a conversa. Sempre reserve um tempo adicional na sua agenda para uma possível extensão na duração da entrevista, caso seja necessário.
- **Reservar os recursos:** geralmente, as entrevistas são realizadas no ambiente do entrevistado e não no seu, então, ao fazer a reserva da agenda, confirme se o local tem todos os recursos que você necessita, como projetor, acesso à internet etc. Caso os temas a serem tratados sejam confidenciais ou de acesso restrito, certifique-se de que a sala que será usada tem privacidade suficiente.

Uma vez que tudo esteja preparado, agora é só **realizar a entrevista.** Embora estejamos falando "só realizar", essa tarefa não é simples, mesmo com todo o preparo prévio que discutimos. Mais adiante neste capítulo, vamos ver algumas dicas sobre problemas que podem surgir durante a entrevista e como tratá-los.

Uma vez que a entrevista tenha sido realizada e que as informações tenham sido coletadas, é hora de **organizar os requisitos** que foram extraídos. Trataremos deste assunto nas próximas seções.

Observação

Existem processos de negócio que são muito complexos e cuja compreensão é mais bem realizada observando-se sua execução. A observação é uma técnica de elicitação de requisitos que pode ser utilizada quando temos acesso ao ambiente onde o fenômeno de interesse ocorre e temos disponibilidade de tempo para realizá-la. Geralmente, ela é utilizada após o contexto ter sido compreendido por meio de outra técnica, como a entrevista, por exemplo.

De acordo com Wiegers e Beatty (2013), uma observação pode ser de dois tipos, listados a seguir:

- **Silenciosa (sem interação com as pessoas que executam o processo):** as observações silenciosas são usadas quando o usuário não pode ser interrompido durante a realização da tarefa.
- **Interativa (com interrupções para fazer perguntas para quem está realizando a tarefa):** as observações interativas são mais ricas, pois permitem identificar por que, por exemplo, o usuário tomou tal decisão naquele momento. No entanto, é preciso tomar cuidado para que ela não se torne muito invasiva.

Assim como as demais técnicas, a observação requer do analista de requisitos uma preparação adequada, de modo que o tempo dispendido em campo seja proveitoso e as informações relevantes possam ser coletadas. A Figura 3 apresenta as etapas da observação.



Figura 3. Passos para a utilização da técnica de observação.

A primeira atividade a ser realizada é **identificar o processo** que será observado. Isso pode ser feito após a conversa com o patrocinador do projeto e a identificação do contexto no qual o projeto está sendo desenvolvido. Pode ser que apenas uma parte do processo precise ser observada, ou pode ser que o processo todo precise ser acompanhado. A necessidade de observação pode ser identificada após a aplicação de alguma outra técnica. Por exemplo, após uma rodada de entrevistas, percebe-se que existe uma diferença entre o que está descrito em um processo ou manual e o que as pessoas relataram. A observação ajuda a esclarecer como o processo é realmente praticado.

Outra aplicação da técnica de observação pode ser para avaliar problemas na usabilidade de produtos de *software*. Muitas vezes, o simples relato do problema não é suficiente para que se perceba onde estão os gargalos da interface. Ao observar o usuário utilizando o *software*, esses problemas podem emergir mais facilmente.

Após a identificação do processo-alvo, é necessário **preparar a observação**, ou seja, planejar como ela será realizada. Para que o trabalho em campo seja proveitoso, o analista de requisitos deve iniciar pela aquisição do conhecimento sobre o contexto e o processo em si. Esse conhecimento pode ser obtido pela análise dos documentos que descrevem o processo, se houver, ou por entrevistas anteriores, na qual algum dos *stakeholders* já tenha descrito como as atividades são executadas. Isso permite que o analista de requisitos identifique de antemão se o processo ocorre de forma similar em todos os momentos do dia, se existe algum período de pico, se as variações do processo são previsíveis ou se ocorrem aleatoriamente, se existe alguma situação especial que precise ser observada de forma específica e assim por diante. Isso ajuda a planejar sua agenda, para saber os momentos em que deverá estar presente.

A etapa de preparação inclui o agendamento com a área onde o processo está sendo executado, bem como o esclarecimento dos objetivos da observação. É importante que o analista de requisitos tenha tempo disponível além do planejado na agenda, pois podem ocorrer situações imprevistas que são de interesse da observação e que não estejam, inicialmente, no cronograma planejado.

O analista de requisitos também deve preparar todo o material que julgue necessário para apoiá-lo durante a observação. Isso inclui prancheta para anotações manuais (ou *tablets*), diagramas do processo (se houver), *check-lists* com pontos importantes de observação e cronômetro (que pode ser o do celular ou *tablet*). Caso a observação seja realizada em um ambiente que requeira o uso de equipamentos de proteção individual (EPI), eles deverão ser providenciados com antecedência.



Fique atento

Nenhuma gravação em vídeo ou áudio poderá ser realizada sem autorização prévia das pessoas envolvidas. Se houver a intenção de gravar, essa autorização deve ser obtida com antecedência. Mas lembre-se de que, quando as pessoas sabem que estão sendo gravadas, seu comportamento pode ser alterado e isso pode introduzir um viés nos resultados da observação.

A próxima atividade é **realizar a observação**. Alguns cuidados devem ser tomados para que a observação ocorra com sucesso, e isso será detalhado na próxima seção.

Após concluída a observação, o analista de requisitos deve **analisar as informações** coletadas, confrontando-as com as que já tiverem sido levantadas em momentos anteriores. Pode ser que algumas informações sejam confirmadas pelas observações em campo, e pode ser que outras sejam diferentes. Nesse caso, é necessário que seja definido como os requisitos provenientes dessas inconsistências serão tratados, e isso pode envolver a tomada de decisão entre um ou mais *stakeholders* do projeto. Nesse caso, reuniões estruturadas ou *workshops* no formato JAD podem ajudar.

Ao realizar a observação, o analista de requisitos também pode identificar pontos de melhoria nos processos de negócio, e essas sugestões também precisam ser discutidas com os *stakeholders*, pois podem implicar em alterações em requisitos já identificados.

2 Aplicação das técnicas de elicitação de requisitos

As técnicas de elicitação de requisitos podem ser aplicadas de forma isolada ou combinadas, dependendo do contexto em que o projeto está inserido. O Quadro 1, baseado em Dennis, Wixon e Roth (2012), que vimos anteriormente, analisa as seguintes variáveis, para as técnicas mais comuns: tipo de informação que pode ser obtida com a aplicação da técnica, profundidade da informação, abrangência da informação, integração da informação, envolvimento requerido do usuário e custo. Isto pode ajudar o analista de requisitos a optar por uma ou mais técnicas.

Podemos, por exemplo, iniciar com uma entrevista realizada com o patrocinador do projeto, que pode fornecer requisitos relacionados ao processo, ao projeto e ao produto em si. Em seguida, podemos aplicar um questionário para confirmar informações ou para compreender as dores dos futuros usuários do *software*. Depois, talvez precisemos nos aprofundar nas leis ou regulamentações do setor, que podem ser fontes importantes de regras de negócio. Por fim, uma observação de partes específicas do processo pode ser necessária.

Essas decisões são tomadas na etapa de planejamento e aplicadas na fase de execução da elicitação. Quando o analista de requisitos não é muito experiente, outros membros da equipe podem ajudá-lo a tomar essas decisões.



Fique atento

Independentemente da técnica a ser utilizada, uma série de habilidades são importantes para que o analista de requisitos possa realizar suas atividades de elicitação de requisitos com sucesso. Wiegers e Beatty (2013) citam algumas, listadas a seguir:

- habilidade de escutar;
- habilidade de entrevistar e questionar;
- capacidade de pensar/agir rapidamente;
- capacidade de aprender;
- habilidade de comunicação;
- habilidade de organização;
- habilidade de modelagem;
- habilidade analítica;
- habilidades interpessoais;
- pensamento sistêmico;
- liderança;
- criatividade;
- observação;
- ser um facilitador.

As próximas seções apresentam algumas dicas para o momento de aplicação das três técnicas tratadas neste capítulo: questionário, entrevista e observação. Muitas dessas dicas servem para outras técnicas de elicitação de requisitos.

Questionário

Uma vez que o questionário tenha sido preparado com todos os cuidados que citamos, o próximo grande problema pode ser a pequena quantidade de respostas. Responder questionários não é uma atividade prazerosa, e a maioria das pessoas não gosta de fazê-lo. Por isto é comum que sejam oferecidas duas formas de recompensa pelo tempo dispendido: uma é enviar os resultados agregados da pesquisa, quando eles estiverem concluídos; a outra é oferecer algum tipo de benefício para o respondente, que pode variar de acordo com o ramo de atuação. Esse benefício pode ser o acesso gratuito ao produto que está sendo desenvolvido, por X meses, por exemplo. Pode ser ainda algum item de pequeno valor, como uma caneta.

Outra forma de aumentar a quantidade de respondentes é usar a técnica de *snowballing*, na qual você pede para que o respondente repasse o questionário a seus conhecidos e que estes também repassem para os seus conhecidos. Isso pode ser útil se você não precisa ter controle de quem vai responder. Caso o seu público-alvo seja muito específico, então esta técnica pode não ser aplicada.

Quando o questionário é aplicado para um público interno da empresa, uma boa estratégia pode ser pedir o apoio das gerências envolvidas, fazendo com que sejam as promotoras do questionário.

Há situações em que o questionário será aplicado na forma de entrevista estruturada. Nesses casos, é importante que as pessoas que irão aplicar o questionário sejam treinadas para fornecer sempre as mesmas explicações para os respondentes, pois, caso essas explicações sejam diferentes, os resultados podem ser invalidados.

Entrevista

A entrevista é, possivelmente, a técnica de elicitação de requisitos mais utilizada no mundo e também a que mais exige em termos de habilidades e experiência do analista de requisitos. Um planejamento bem executado, seguindo as orientações da seção anterior, já é meio caminho andado, mas a execução em si enseja diversos desafios.

A primeira questão que pode surgir quando a entrevista iniciar é se é possível gravar para poder escutar novamente depois. Com a facilidade de gravação e a maior capacidade de armazenamento dos celulares, isso se tornou bastante comum. Entretanto, nem todas as pessoas se sentem à vontade com gravações. É necessário ter autorização para gravar. Mesmo gravando, faça suas próprias anotações, pois *insights* podem surgir durante a fala do entrevistado e eles precisam ser registrados no momento em que surgem, pois é grande a chance de esquecê-los depois.



Fique atento

Jamais faça uma gravação sem que o entrevistado saiba e tenha autorizado.

Uma das habilidades mais importantes para realizar com sucesso uma entrevista é a comunicação. A comunicação se dá em duas vias: falar e escutar. Tão importante quanto saber fazer as perguntas certas é saber escutar ativa e atentamente o que é dito. Muitas informações são passadas nas entrelinhas e ficam implícitas durante a entrevista. E aí reside um grande problema para a elicitação de requisitos, pois nada deve permanecer implícito. Uma dica é buscar retomar os pontos principais, verificando se o que foi entendido era realmente o que o entrevistado queria dizer. Também não é preciso ter vergonha de retomar um tema ou um termo que não ficou muito claro. Melhor perguntar novamente na hora do que gerar requisitos incompletos ou incorretos depois.

Comunicar-se implica em se fazer entender pelo outro lado. Para que o entrevistado compreenda corretamente o que está sendo perguntado e discutido, é importante que o analista de requisitos utilize sentenças mais curtas e claras, sem jargões técnicos que possam ser incompreensíveis para o usuário. Figuras ou diagramas de apoio podem ajudar.

De acordo com Wiegers (2006), a pior pergunta pela qual o analista de requisitos pode iniciar a entrevista é "O que você quer?". E a segunda pior é "Quais são os seus requisitos?". Ninguém sabe como responder a essas perguntas, e pode ser que o entrevistado nem entenda exatamente o que é um requisito. A resposta, provavelmente, será uma lista aleatória dos pensamentos que vierem na cabeça dele. Procure iniciar perguntando que problemas de negócio ele pretende resolver e porque isso é importante. Essa será a porta de entrada para o entrevistado contar sobre os processos de negócio e como o sistema poderá ajudá-lo.

Uma dica para aproveitar ao máximo o tempo da entrevista é utilizar a técnica do "e se…". Para cada parte do processo relatada, busque explorar as opções "e se…", ou seja, "e se não tiver o cadastro completo, o que acontece?", "e se não houver comunicação *on-line* no momento do pagamento, o que o sistema deve fazer?". Isso é importante para identificar os caminhos alternativos do processo e como o sistema deve se preparar para tratá-los. É comum que o usuário só lembre de falar sobre o caminho principal de um fluxo de processo.

Outra habilidade importante no momento da condução da entrevista é saber pensar rápido e achar uma saída, quando uma situação inesperada surge. É o que os americanos chamam de "*thinking on your feet"*. Por mais bem construído que tenha sido o seu roteiro, você precisa estar preparado para lidar com situações não previstas. Infelizmente, isso não tem como ser ensinado. Vai vir com a experiência e com o domínio do contexto do problema.



Fique atento

Uma questão bastante comum em entrevistas é o vai e vem entre níveis distintos de abstração, ou seja, quando o entrevistado ora fala em um processo de alto nível e ora fala de um detalhe pequeno sobre uma tarefa específica. Cabe ao analista de requisitos manter a conversa no nível de abstração necessário para identificar os requisitos do *software* e navegar entre eles com habilidade para esclarecer todas as suas dúvidas. A visão sistêmica também lhe será útil para abstrair as especificidades de cada usuário em particular, enxergando o panorama geral.

Observação

Assim como a entrevista, a observação exige do analista de requisitos algumas habilidades especificas. A principal delas, naturalmente, é a capacidade de observar. Observar requer atenção aos detalhes e às questões subjetivas ou implícitas. Um bom analista de requisitos consegue identificar detalhes na execução das tarefas não mencionados anteriormente em uma entrevista, por exemplo.

Quando a observação é ativa, ou seja, é possível interagir com as pessoas que executam as tarefas, cuidados adicionais devem ser tomados. Por exemplo, deve-se evitar interromper a realização da tarefa com muita frequência, pois isso pode desviar a atenção do trabalhador da sua execução, atrapalhar o processo e introduzir um viés na própria observação. As perguntas devem ser pontuais e estritamente relacionadas com as tarefas realizadas no momento. Se as dúvidas forrem de outra natureza, então, talvez seja necessário agendar uma entrevista.

3 Organização dos requisitos de *software*

Quanto mais conhecemos a respeito dos requisitos, maiores são as chances de sucesso do projeto. Quando o analista de requisitos inicia o seu trabalho, pode ser que ele se depare com uma mistura de requisitos de processo, de projeto e de produto. Isso ocorre, principalmente, nas entrevistas, uma vez que, para o entrevistado, tudo o que ele está falando diz respeito ao projeto que está

sendo desenvolvido. Ele vê tudo como uma coisa só, não separando produto, processo e projeto. Mas para a equipe de desenvolvimento, é necessário que esses requisitos sejam separados, de forma a serem tratados adequadamente por cada papel no ciclo de desenvolvimento.

Os **requisitos de processo** dizem respeito à forma como o produto será desenvolvido. Podem estar relacionados ao uso de determinadas metodologias, padrões, orientações etc. Um cliente que contrata uma organização para desenvolver um produto de *software* pode ter exigências em relação a como esse produto será entregue. Por exemplo, pode ser que ele defina a periodicidade das entregas, a forma como deseja acompanhar a evolução do projeto, o padrão gráfico para a interface, as tecnologias que serão empregadas e até mesmo se a empresa ou os profissionais devem ter uma certificação.

Esses requisitos não farão parte do produto em si, mas irão influenciar a forma como a equipe irá se organizar para o trabalho e até mesmo como ela deverá se preparar para isso. Por exemplo, se for requerido o uso de uma tecnologia que a empresa não tem conhecimento, será necessário adquirir esse conhecimento, seja por meio de treinamento dos profissionais existentes, seja pela contratação de mão de obra especializada. Isso terá um custo que deverá estar planejado e disponibilizado.

Quando a empresa desenvolvedora tiver um processo de desenvolvimento definido e institucionalizado e a empresa contratante requisitar alguma forma diferente de desenvolvimento, a equipe de processos da organização desenvolvedora deverá analisar essas mudanças e realizar as adaptações necessárias no processo padrão em função das exigências do contrato, além de treinar as suas equipes no uso desse novo processo. Por esse motivo, as mudanças no processo precisam ser planejadas e sincronizadas com o cronograma do projeto.

Os **requisitos de projeto** geralmente estão relacionados com aspectos que impõem restrições à forma como o projeto será planejado e executado. Normalmente, dizem respeito às restrições de prazos e custos. Esses requisitos estarão refletidos no cronograma e/ou no orçamento do projeto.

Restrições de prazo podem ser impostas por um *stakeholder* externo, como, por exemplo, um órgão regulamentador que impõe que determinada legislação, ou determinada regulamentação, deva estar implementada até uma data específica. Portanto, para que a organização atenda a essa determinação, o produto de *software* envolvido deve estar adaptado no prazo legalmente imposto.

Outro aspecto que aparece na forma de requisito de projeto é a limitação de orçamento. Muitas vezes a equipe de vendas já fechou o contrato com o cliente e já vendeu o projeto por um valor que não pode ser ultrapassado. Com isso, as opções do gerente de projetos talvez sejam limitadas por essa restrição orçamentária, o que pode influenciar em todos os elementos que demandem custos para o projeto: instalações, tecnologia, equipamentos, pessoas, serviços etc.

Os requisitos de projeto se desdobram em elementos do **plano de projeto** ou artefato similar. No caso dos métodos ágeis, eles se desdobram no planejamento das *sprints* e na alocação de pessoas e recursos.

Os **requisitos de produto**, como o próprio nome diz, são aqueles que, efetivamente, serão implementados no produto de *software*. Eles serão a fonte a partir da qual serão realizadas as estimativas de esforço e tempo, serão desdobradas as especificações técnicas, o código e os casos de teste. Quanto mais bem compreendidos estiverem esses requisitos, maiores as chances de sucesso e menor o retrabalho. Isso não quer dizer que tenhamos que gastar um tempo infinito elicitando e documentando os requisitos, mas, sim, que precisamos ter as informações suficientes para que as próximas etapas possam ocorrer.

Um requisito tem determinados atributos (também chamados de propriedades) que podem ser necessários, dependendo do contexto. O Quadro 2 apresenta algumas dessas propriedades.

Quadro 2. Atributos de um requisito

Atributos	Definição	Possíveis formas de avaliar
Estabilidade	Chances que um requisito tem, ou não, de mudar ou o nível de maturidade de sua especificação. Quanto mais estável um requisito, menos chances ele tem de mudar; quanto menos estável, ou seja, mais volátil, maiores são as chances de ele mudar.	■ Alta, média, baixa
Impacto na arquitetura	Impacto que o requisito tem na arquitetura da aplicação. Quanto maior o impacto na arquitetura, mais crítico é o requisito e mais atenção deve ser dada a ele.	■ Alto, médio, baixo

(Continua)

(Continuação)

Quadro 2. Atributos de um requisito

Atributos	Definição	Possíveis formas de avaliar
Prioridade	Prioridade na implementação do requisito. Pode-se ter dois tipos de prioridade: aquela estabelecida pelo negócio (traz mais valor para o cliente) e aquela estabelecida pela equipe técnica (é o requisito mais viável de ser implementado antes, muitas vezes por restrições técnicas de precedência).	■ Alta, média, baixa ■ Mandatório, altamente desejável, desejável e opcional
Impacto da não implementação	Grau de impacto no negócio, caso o requisito não seja implementado.	■ Alto, médio, baixo ■ Valores financeiros ■ Escala numérica
Dificuldade	Grau de dificuldade para implementação do requisito.	■ Alta, média, baixa
Risco	Grau de risco que o requisito representa para o projeto. O risco pode advir de diversos fatores, como não haver pessoas especialistas para definir ou para implementar o requisito, por exemplo.	■ Alto, médio, baixo
Situação	*Status* de um requisito em um determinado momento.	■ Identificado, priorizado, alocado, implementado, testado, homologado, entregue

Os atributos serão úteis para a definição da priorização dos requisitos, bem como para a sua alocação a uma entrega (versão ou *sprint*). Caso um requisito apresente grande **estabilidade**, significa que ele já está maduro para ser encaminhado para as próximas atividades. Geralmente, requisitos mais estáveis são aqueles que são inerentes ao domínio da aplicação. Caso contrário, ou seja, caso um requisito apresente uma grande volatilidade, significa que ele

não está claramente definido por algum motivo, e será mais prudente que ele seja alocado em uma versão ou *sprint* posterior, ou que seja dedicada especial atenção para uma implementação que permita essa flexibilização.

Identificar os requisitos que tenham alto **impacto na arquitetura** é uma tarefa fundamental. Esses requisitos devem receber especial atenção tanto do analista de requisitos quanto dos projetistas e desenvolvedores. Esse é o tipo de requisito que é muito difícil de retificar posteriormente, pois seu impacto na solução é relevante.

A **prioridade** tem dois aspectos: a prioridade sobre a ótica do negócio e a prioridade técnica. A primeira pode ser definida pelos *stakeholders* que representam o negócio e é derivada da sua contribuição aos processos de negócio. A segunda é definida pela equipe técnica e leva em consideração os aspectos relacionados à viabilidade de implementação. Muitas vezes, por mais prioritário que seja um requisito, ele depende que outros sejam implementados antes.

Para ajudar na definição da prioridade, pode-se utilizar o atributo que avalia o **impacto da não implementação** do requisito, cuja análise é realizada pelos *stakeholders* relacionados ao negócio. Também pode-se utilizar o grau de **dificuldade**, que é um aspecto relacionado à parte técnica.



Saiba mais

O *Software Engineering Body of Knowledge* (SWEBOK), documento que descreve os conhecimentos geralmente aceitos na engenharia de *software*, sugere o uso das seguintes expressões para indicar a prioridade de um requisito: **mandatório**, **altamente desejável**, **desejável** e **opcional** (BOURQUE; FAIRLEY, 2014).

O **risco** é um atributo que avalia qual é o grau de risco do requisito. Ele pode ser derivado da análise conjunta das demais propriedades e também pode estar relacionado a questões específicas, como a indisponibilidade de pessoas com experiência para implementá-lo.

Algumas combinações entre as propriedades requerem uma atenção especial. Vamos analisar um caso no qual um requisito é de alta prioridade para o negócio, tem um grande impacto na arquitetura, mas ainda não está estável. Isso significa que ele deve ser analisado com mais cuidado. Talvez seja necessário gerenciá-lo de forma especial, envolvendo os *stakeholders* que

podem solucionar a instabilidade e definindo, por exemplo, provas de conceito com a arquitetura candidata.

Cada requisito, em um dado momento, está em uma situação ou *status*. A organização pode definir quais estados ela deseja controlar. É importante lembrar que, quanto mais estados possíveis, maior o esforço de controle, pois cada mudança de estado de cada requisito precisará ser registrada e atualizada. Exemplos de estados podem ser, mas não se resumem a: identificado, priorizado, alocado, especificado, implementado, testado, homologado e entregue.



Saiba mais

Nos métodos ágeis, é comum a utilização de histórias de usuários (*user stories*), que podem ser entendidas como as intenções que o usuário tem com o sistema. De acordo com Leffingwell (2011), as diferenças entre as histórias de usuário e outras abordagens (como o documento de especificação de requisitos) são as apresentadas a seguir.

- Não são especificações detalhadas de requisitos (alguma coisa que o sistema deve fazer), mas são expressões negociáveis de intenção.
- São curtas, fáceis de ler e compreensíveis pelos desenvolvedores, *stakeholders* e usuários.
- Representam pequenos incrementos de funcionalidade de valor, que podem ser desenvolvidos em um período de dias a semanas.
- São relativamente fácies de estimar, então, o esforço para implementar a funcionalidade pode ser rapidamente determinado.
- Não são descritas em documentos longos e pesados, e sim são organizadas em listas que podem ser facilmente organizadas e reorganizadas à medida que novas informações são descobertas.
- Não são detalhadas logo no início do projeto, são elaboradas no momento em que são requeridas, evitando especificidades muito cedo no projeto, atrasos no desenvolvimento, inventário de requisitos e uma declaração super restrita da solução.
- Precisam de pouca ou nenhuma manutenção e podem ser descartadas seguramente depois da implementação.
- As histórias de usuário e o código que é criado rapidamente logo em seguida servem como entrada para a documentação, que é então desenvolvida de forma incremental.



Referências

BARRIBEAU, P. *et al*. Survey research. *In*: WRITING@CSU. Colorado, 2020. Disponível em: https://writing.colostate.edu/guides/guide.cfm?guideid=68. Acesso em: 2 fev. 2020.

BOURQUE, P.; FAIRLEY, R. *SWEBOK:* guide to the *software* engineering body of knowledge version 3. [*S. l.*]: IEEE Computer Society, 2014.

DENNIS, A.; WIXON, B.; ROTH, R. *System analysis and design*. 5. ed. Hoboken: John Wiley and Sons, 2012.

LEFFINGWELL, D. *Agile software requirements:* lean requirements practices for teams, programs, and the enterprise. Upper Saddle River: Pearson Education, 2011.

WIEGERS, K. E.; BEATTY, J. *Software requirements*. 3. ed. Redmond: Microsoft Press, 2013.

WIEGERS, K. *More about software requirements*: thorny issues and practical advices. Redmond: Microsoft Press, 2006.

PREZADO ESTUDANTE

ENCERRA AQUI O TRECHO DO LIVRO DISPONIBILIZADO PELA SAGAH PARA ESTA PARTE DA UNIDADE.

CONTRIBUA COM A QUALIDADE DO SEU CURSO

Se você encontrar algum problema neste material, entre em contato pelo email **eadproducao@unilasalle.edu.br.** Descreva o que você encontrou e indique a página.

Lembre-se: a boa educação se faz com a contribuição de todos!

UNIVERSIDADE
LaSalle
Viver é evoluir.

Av. Victor Barreto, 2288 | Canoas - RS
CEP: 92010-000 | 0800 541 8500
eadproducao@unilasalle.edu.br